Lecture 7: Clustering, Gaussian mixtures

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 13, 2026

Logistics

- End of today's class: Quiz 1
- This Wed.: Tutorial 4 (more neural architectures)
- HW2 released, due April 23

HW1 debriefing

Student feedback

- # hours spent $\in [6, 30]$
- Readings are helpful
- Writing down problem formulation before coding it up is helpful
- LLMs useful for explaining missed details in lecture (but note: not clear this produces correct understanding)
- Multiple partitions in CCA experiments confusing (sorry!)

Other notes

- Many solutions are "tutorial-style" – shorter would be better
- Some confusion about CCA formulation

Canonical correlation analysis (CCA) [Hotelling 1936]

- Given: data set of n paired vectors $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, which are samples of random vectors $X \in \mathbf{R}^{d_x}, Y \in \mathbf{R}^{d_y}$
- Find: direction vectors v, w that maximize the correlation between the projections $v^T X$ and $w^T Y$

Canonical correlation analysis (CCA) [Hotelling 1936]

- Given: data set of n paired vectors $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, which are samples of random vectors $X \in \mathbf{R}^{d_x}, Y \in \mathbf{R}^{d_y}$
- Find: direction vectors v, w that maximize the correlation between the projections $v^T X$ and $w^T Y$
- First pair of directions:

$$v_1, w_1 \quad = \quad \underset{v,w}{\operatorname{argmax}} \ \operatorname{corr}(v^T X, w^T Y)$$

Canonical correlation analysis (CCA) [Hotelling 1936]

- Given: data set of n paired vectors $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, which are samples of random vectors $X \in \mathbf{R}^{d_x}, Y \in \mathbf{R}^{d_y}$
- Find: direction vectors v, w that maximize the correlation between the projections $v^T X$ and $w^T Y$
- First pair of directions:

$$\begin{aligned} v_1, w_1 &= \operatorname*{argmax}_{v,w} \operatorname{corr}(v^T X, w^T Y) \\ &= \operatorname*{argmax}_{v,w} \frac{v^T C_{xy} w}{\sqrt{(v^T C_{xx} v)(w^T C_{yy} w)}} \end{aligned}$$

Canonical correlation analysis (CCA) [Hotelling 1936]

- Given: data set of n paired vectors $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, which are samples of random vectors $X \in \mathbf{R}^{d_x}, Y \in \mathbf{R}^{d_y}$
- Find: direction vectors v, w that maximize the correlation between the projections $v^T X$ and $w^T Y$
- First pair of directions:

$$\begin{aligned} v_1, w_1 &= \operatorname*{argmax}_{v,w} \operatorname{corr}(v^T X, w^T Y) \\ &= \operatorname*{argmax}_{v,w} \frac{v^T C_{xy} w}{\sqrt{(v^T C_{xx} v)(w^T C_{yy} w)}} \end{aligned}$$

- Subsequent direction vectors maximize the same correlation, subject to being uncorrelated with the previous directions

Canonical correlation analysis (CCA) [Hotelling 1936]

- Given: data set of n paired vectors $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, which are samples of random vectors $X \in \mathbf{R}^{d_x}, Y \in \mathbf{R}^{d_y}$
- Find: direction vectors v, w that maximize the correlation between the projections $v^T X$ and $w^T Y$
- First pair of directions:

$$\begin{aligned} v_1, w_1 &= \operatorname*{argmax}_{v,w} \operatorname{corr}(v^T X, w^T Y) \\ &= \operatorname*{argmax}_{v,w} \frac{v^T C_{xy} w}{\sqrt{(v^T C_{xx} v)(w^T C_{yy} w)}} \end{aligned}$$

- Subsequent direction vectors maximize the same correlation, subject to being uncorrelated with the previous directions
- Equivalent to maximizing the covariance, subject to $v^T C_{xx} v = 1, w^T C_{yy} w = 1$ (note: vs and ws not orthonormal!)

Canonical correlation analysis (CCA) [Hotelling 1936]

- Given: data set of n paired vectors $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, which are samples of random vectors $X \in \mathbf{R}^{d_x}, Y \in \mathbf{R}^{d_y}$
- Find: direction vectors v, w that maximize the correlation between the projections $v^T X$ and $w^T Y$
- First pair of directions:

$$\begin{aligned} v_1, w_1 &= \underset{v,w}{\operatorname{argmax}} \ \operatorname{corr}(v^T X, w^T Y) \\ &= \underset{v,w}{\operatorname{argmax}} \ \frac{v^T C_{xy} w}{\sqrt{(v^T C_{xx} v)(w^T C_{yy} w)}} \end{aligned}$$

- Equivalent to maximizing the covariance, subject to $v^T C_{xx} v = 1, w^T C_{yy} w = 1$
- Subsequent direction vectors maximize the same correlation, subject to projections being uncorrelated with previous projections: $\operatorname{corr}(v_i^T X, v_j^T X) = 0, i \neq j$, and same for ws (note: vs and ws not orthonormal!)

Canonical correlation analysis (CCA)

Alternative formulations: Now let $X \in \mathbf{R}^{n \times d_x}, Y \in \mathbf{R}^{n \times d_y}$ be data matrices (samples) for the two views. Find $V \in \mathbf{R}^{d_x \times k}, W \in \mathbf{R}^{d_y \times k}$ that

$$(1) \qquad \text{minimize}: \quad E||W^T Y - V^T X||_F^2$$

$$\text{subject to}: \quad V^T C_{xx} V = W^T C_{yy} W = I$$

Canonical correlation analysis (CCA)

Alternative formulations: Now let $X \in \mathbf{R}^{n \times d_x}, Y \in \mathbf{R}^{n \times d_y}$ be data matrices (samples) for the two views. Find $V \in \mathbf{R}^{d_x \times k}, W \in \mathbf{R}^{d_y \times k}$ that

$$\begin{aligned} & (1) \quad \text{minimize}: && E||W^T Y - V^T X||_F^2 \\ OR \ & (2) \quad \text{maximize}: && \operatorname{Tr} V^T C_{xy} W \\ & \quad\ \ \text{subject to}: && V^T C_{xx} V = W^T C_{yy} W = I \end{aligned}$$

Plan for today

- Clustering, hierarchical clustering, spectral clustering
- Gaussian mixtures?
- Quiz 1

Clustering

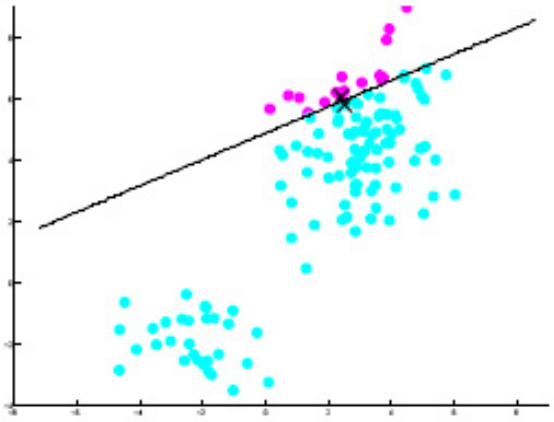
Clustering

- Many data sets naturally consist of multiple natural subsets, e.g. multiple digits, object classes in images, topics, phonemes in speech, ...
- Clustering = the problem of identifying these subsets and assigning data points to the correct one
- One of the key tasks in unsupervised learning and data analysis
- **Warning:** Almost no theoretical guarantees ☹. Treat all clustering results with some skepticism.

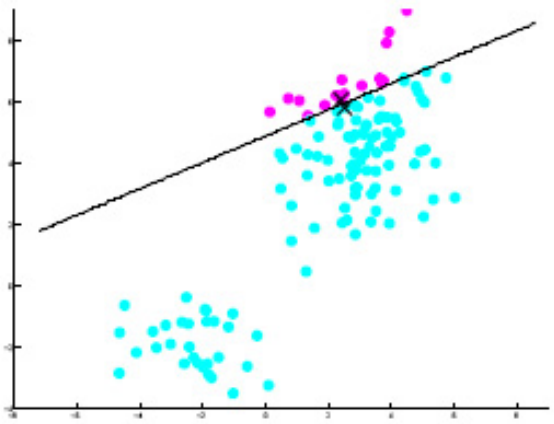
K-means clustering

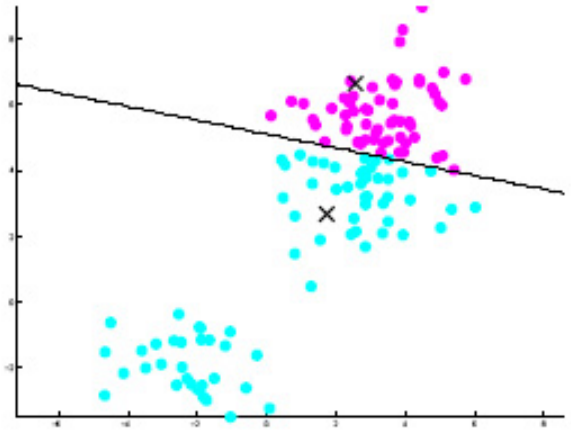
- Simplest VQ approach: k-means clustering
- Algorithm: given number of codewords (clusters) k and a set of examples,
  1. Initialize k means $\mu_1, \ldots, \mu_k$ to random locations
  2. Assign each example $\mathbf{x}_i$ to the closest mean: $y_i = \text{argmin}_c ||\mathbf{x}_i - \mu_c||$
  3. Reestimate each mean based only on examples assigned to it: $\mu_c = \frac{1}{N_c} \sum_{y_i=c} \mathbf{x}_i$, where $N_c = |\{\mathbf{x}_i : y_i = c\}|$
  4. Repeat 2 and 3 until there are no changes in assignment

K-means example with $k = 2$

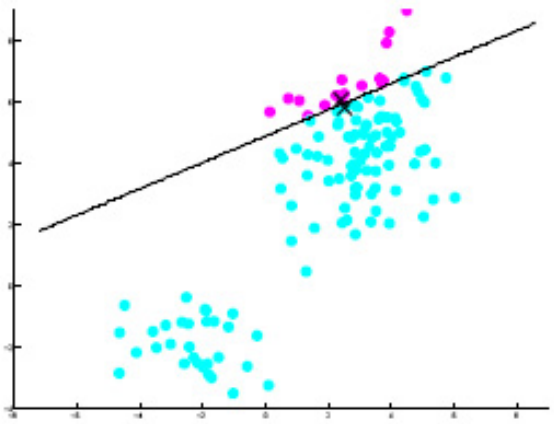


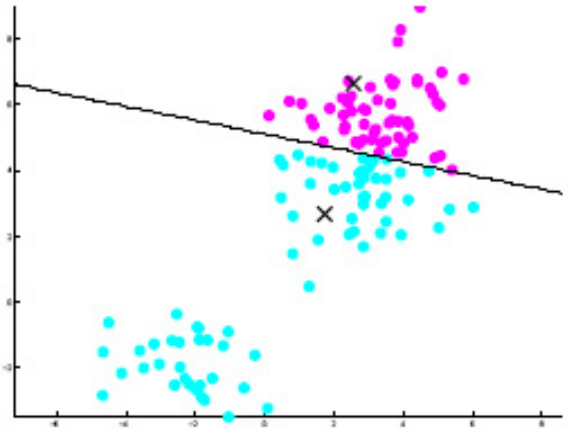
K-means example with $k = 2$

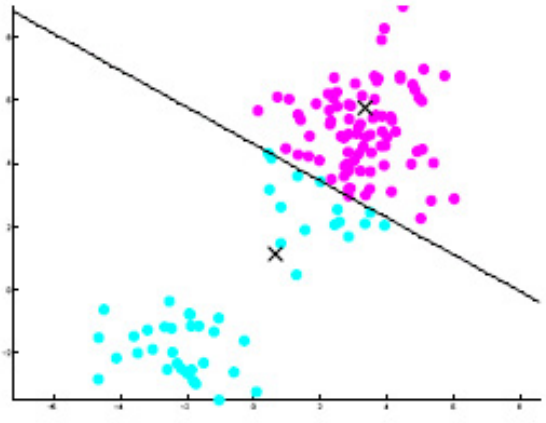




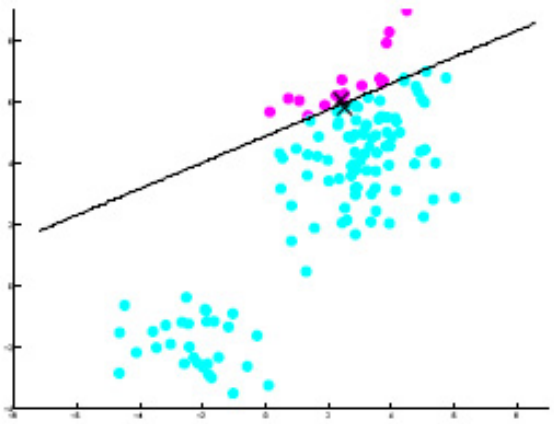
K-means example with $k = 2$

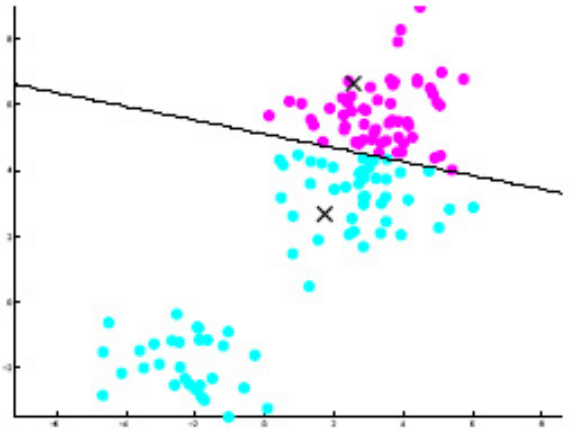


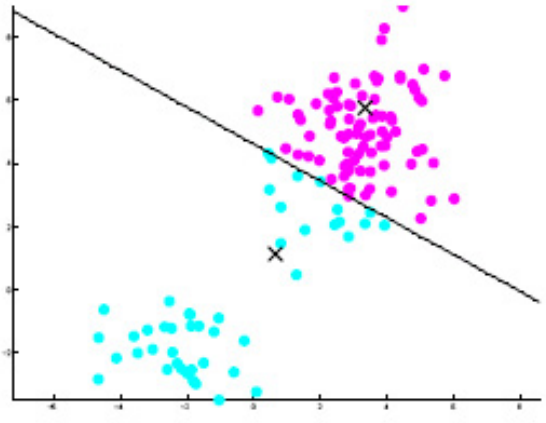


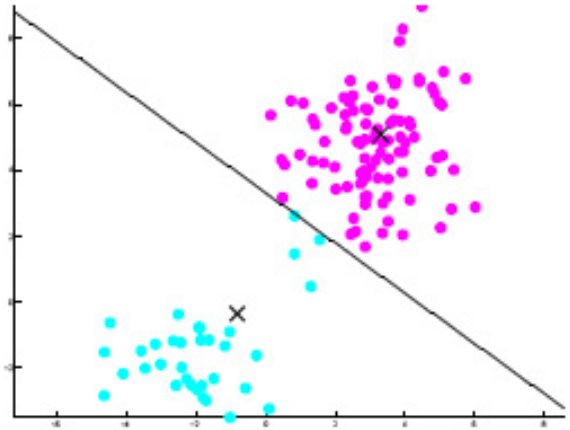


K-means example with $k = 2$

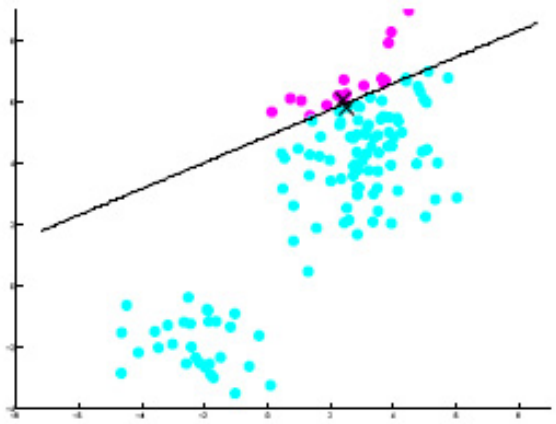


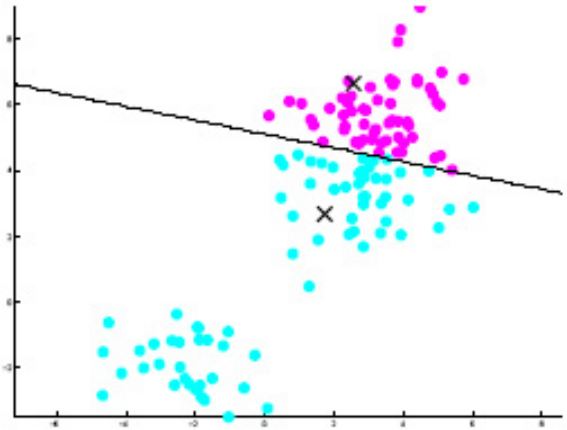


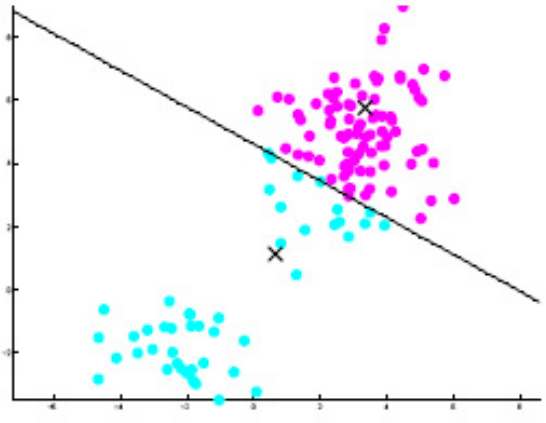


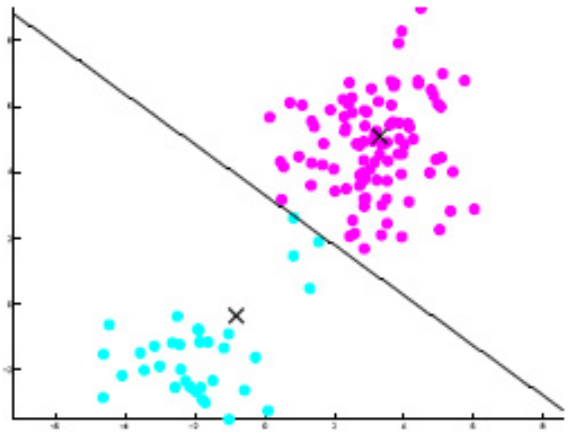


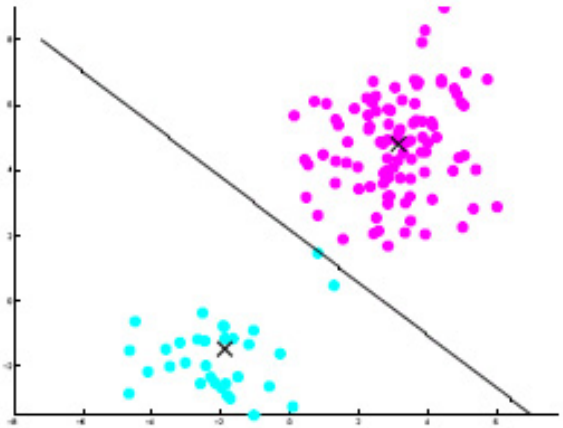
K-means example with $k = 2$



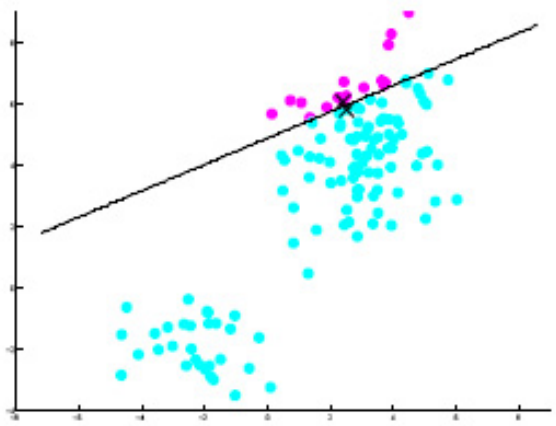
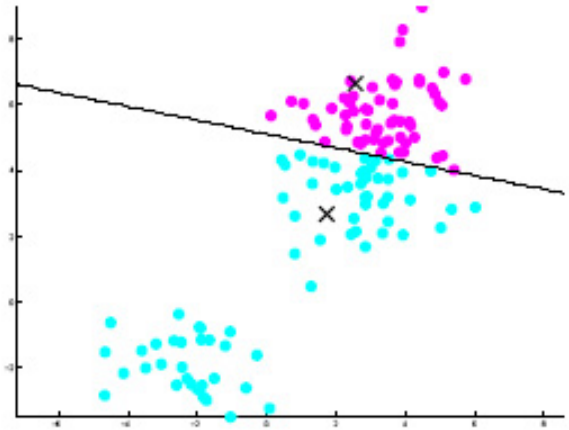
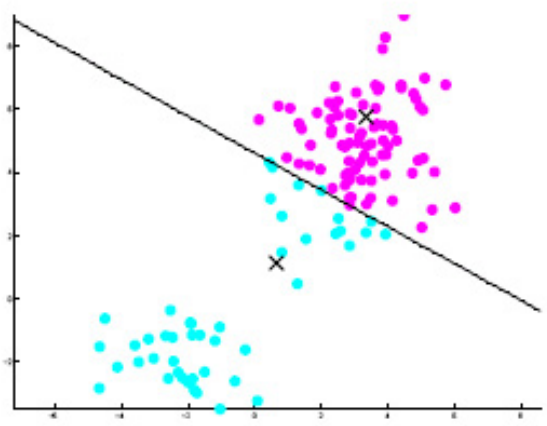
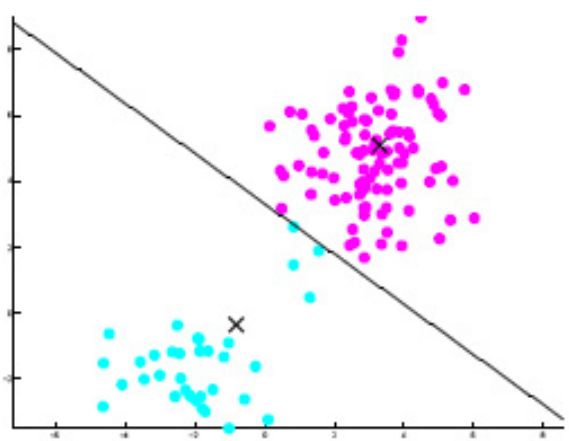
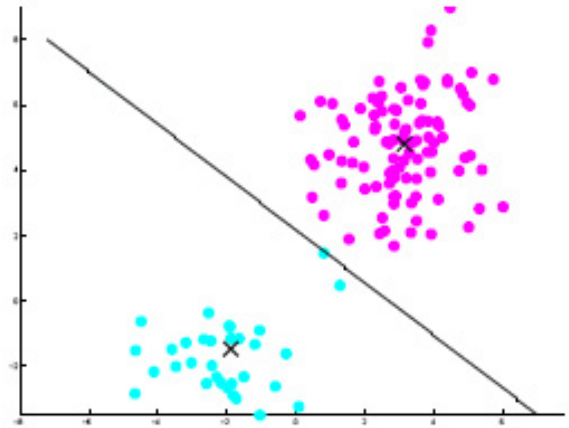
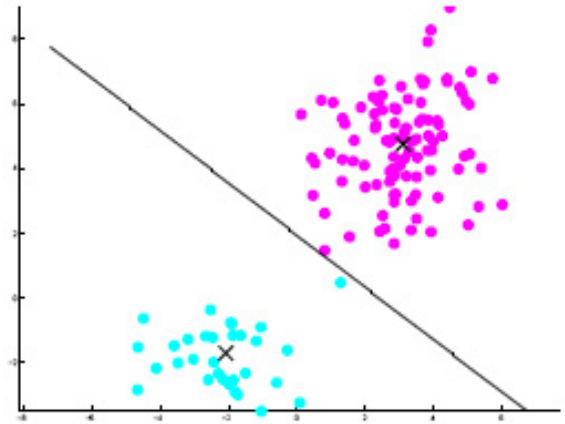








K-means example with $k = 2$

K-means properties

- Total distortion $D = \sum_{c=1}^{k} \sum_{\mathbf{x} \in \text{cluster } c} ||\mathbf{x} - \mu_c||^2$
- D decreases from one iteration to the next
- K-means converges to a local, but not necessarily global, minimum of D
- Initialization can affect final result
- Heuristic remedy: Run k-means with several initializations; keep the result with lowest D

K-means properties

- Total distortion $D = \sum_{c=1}^{k} \sum_{\mathbf{x} \in \text{cluster } c} ||\mathbf{x} - \mu_c||^2$
- D decreases from one iteration to the next
- K-means converges to a local, but not necessarily global, minimum of D
- Initialization can affect final result
- Heuristic remedy: Run k-means with several initializations; keep the result with lowest D
- Choice of distance measure interacts with cluster shape:



Hierarchical clustering

- Agglomerative (bottom-up): Start with each data point as its own cluster, iteratively combine to build increasingly large clusters
- Top-down: Start with all data in a single cluster, iteratively divide to build smaller and smaller clusters

Hierarchical clustering

- Agglomerative (bottom-up): Start with each data point as its own cluster, iteratively combine to build increasingly large clusters
- Top-down: Start with all data in a single cluster, iteratively divide to build smaller and smaller clusters

Why hierarchical clustering?

Hierarchical clustering

- Agglomerative (bottom-up): Start with each data point as its own cluster, iteratively combine to build increasingly large clusters
- Top-down: Start with all data in a single cluster, iteratively divide to build smaller and smaller clusters

Why hierarchical clustering?

- Sometimes there is a natural hierarchy (e.g. biological taxonomies, Wikipedia topics, ...)
- Sometimes it is easier to learn a good (flat) clustering via a hierarchical approach

Agglomerative clustering

- Start with each item in cluster by itself, then iterate:
  - Find "closest" pair of clusters
  - Merge them into single cluster

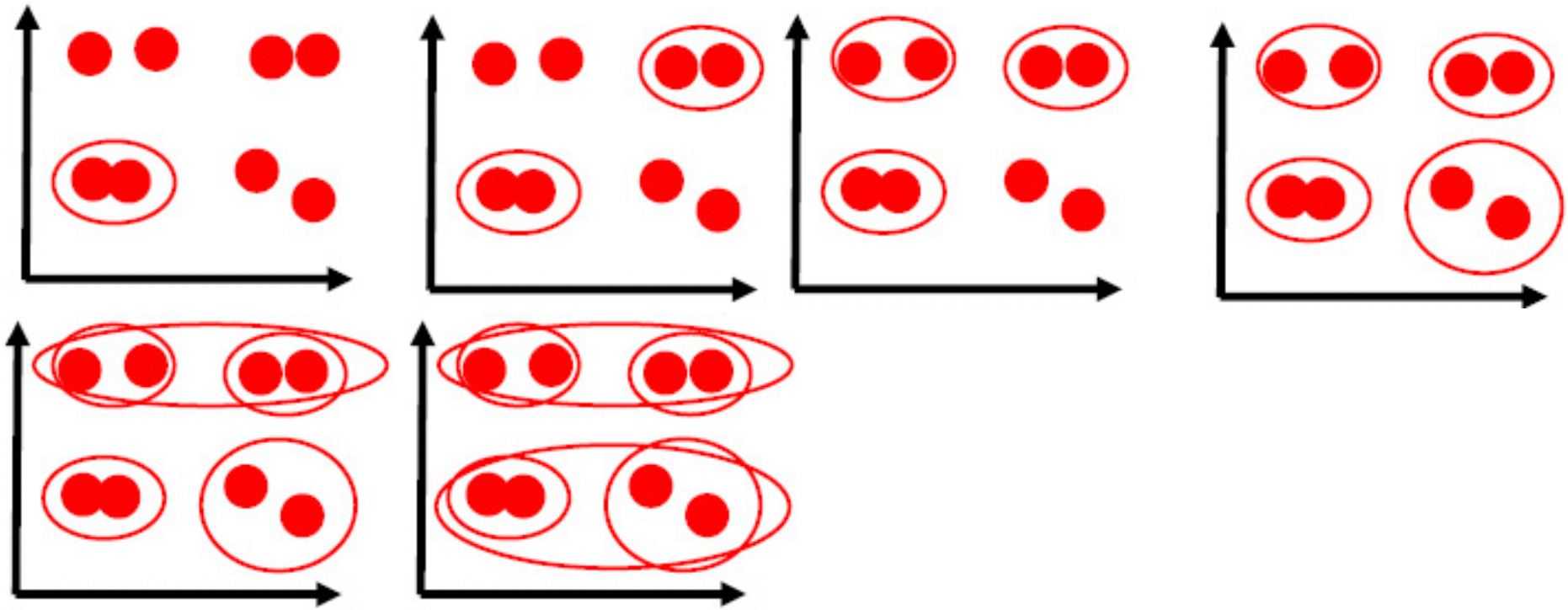
Agglomerative clustering

- Start with each item in cluster by itself, then iterate:
  - Find "closest" pair of clusters
  - Merge them into single cluster
- Until... some convergence criterion
  - Distance between closest clusters is above some threshold
  - Number of data points in each cluster is above some threshold

Agglomerative clustering

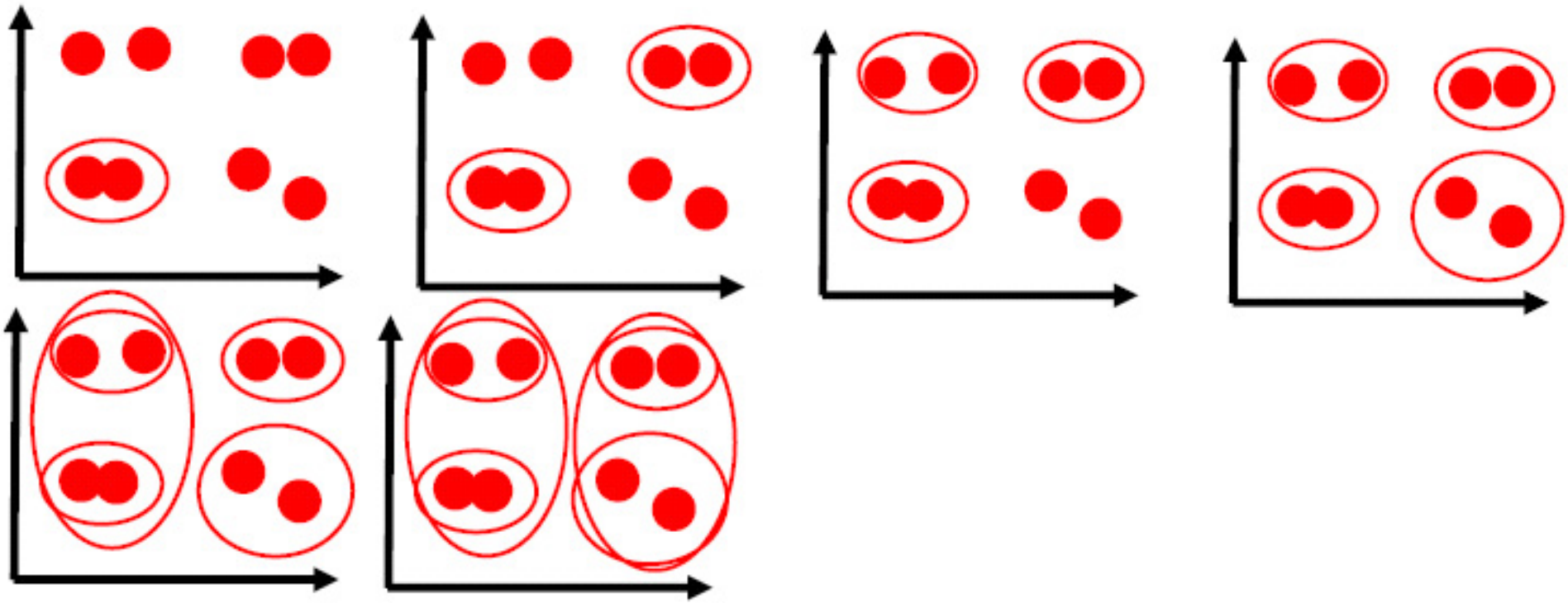
- Start with each item in cluster by itself, then iterate:
  - Find "closest" pair of clusters
  - Merge them into single cluster
- Until... some convergence criterion
  - Distance between closest clusters is above some threshold
  - Number of data points in each cluster is above some threshold
- Different clusterings based on distance measure:
  - *Single-linkage*: $\mathrm{dist}(A, B) = \min_{a \in A, b \in B} \mathrm{dist}(a, b)$
  - *Complete-linkage*: $\mathrm{dist}(A, B) = \max_{a \in A, b \in B} \mathrm{dist}(a, b)$
  - *Average-linkage*: $\mathrm{dist}(A, B) = \mathrm{mean}_{a \in A, b \in B} \mathrm{dist}(a, b)$
  - where a and b are individual data points and A and B are clusters

Example: Single-linkage clustering



Tends to yield long, "stringy" clusters

Example: Complete-linkage clustering



Tends to yield compact, "round" clusters

Ward's method

An agglomerative clustering approach

- Closely related to linkage methods: iterative method that merges the two clusters with minimum distance at each step
- Requires a vector representation of each data point $\mathbf{x}_i$, not just a distance matrix
- For each current cluster A, define the cluster mean $\mathbf{m}_A = \frac{1}{|A|}\sum_{i \in A} \mathbf{x}_i$
- Distance between two clusters A, B defined as **how much the sum of squared distances to the mean will increase if we merge them**:

$$\begin{aligned} \text{dist}(A,B) &= \sum_{i \in A \cup B} ||\mathbf{x}_i - \mathbf{m}_{A\cup B}||^2 - \sum_{i \in A} ||\mathbf{x}_i - \mathbf{m}_A||^2 - \sum_{i \in B} ||\mathbf{x}_i - \mathbf{m}_B||^2 \\ &= \frac{|A||B|}{|A|+|B|} ||\mathbf{m}_A - \mathbf{m}_B||^2 \end{aligned}$$

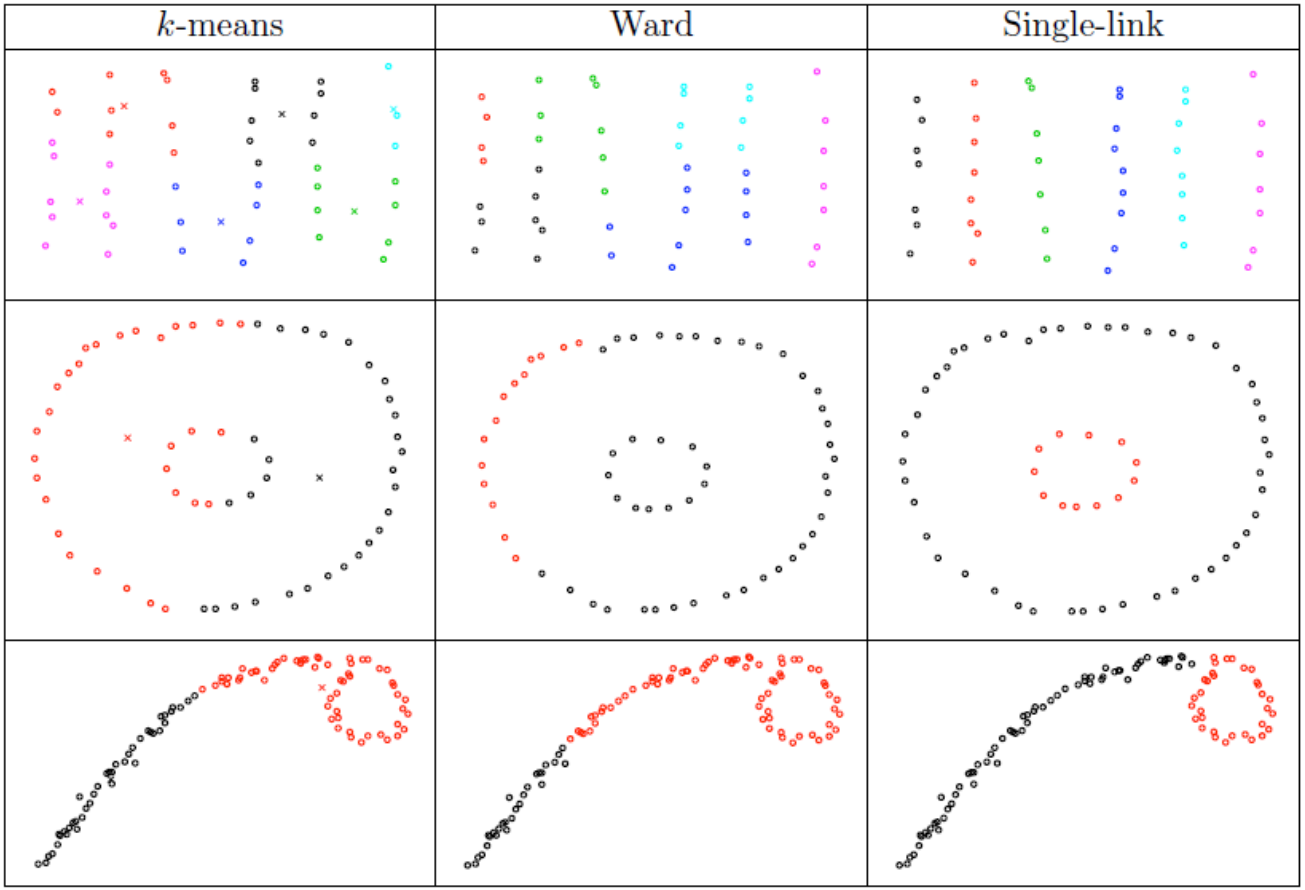
Ward's method

$$\begin{aligned}\text{dist}(A,B) &= \sum_{i\in A\cup B} ||\mathbf{x}_i - \mathbf{m}_{A\cup B}||^2 - \sum_{i\in A} ||\mathbf{x}_i - \mathbf{m}_A||^2 - \sum_{i\in B} ||\mathbf{x}_i - \mathbf{m}_B||^2 \\ &= \frac{|A||B|}{|A|+|B|} ||\mathbf{m}_A - \mathbf{m}_B||^2\end{aligned}$$

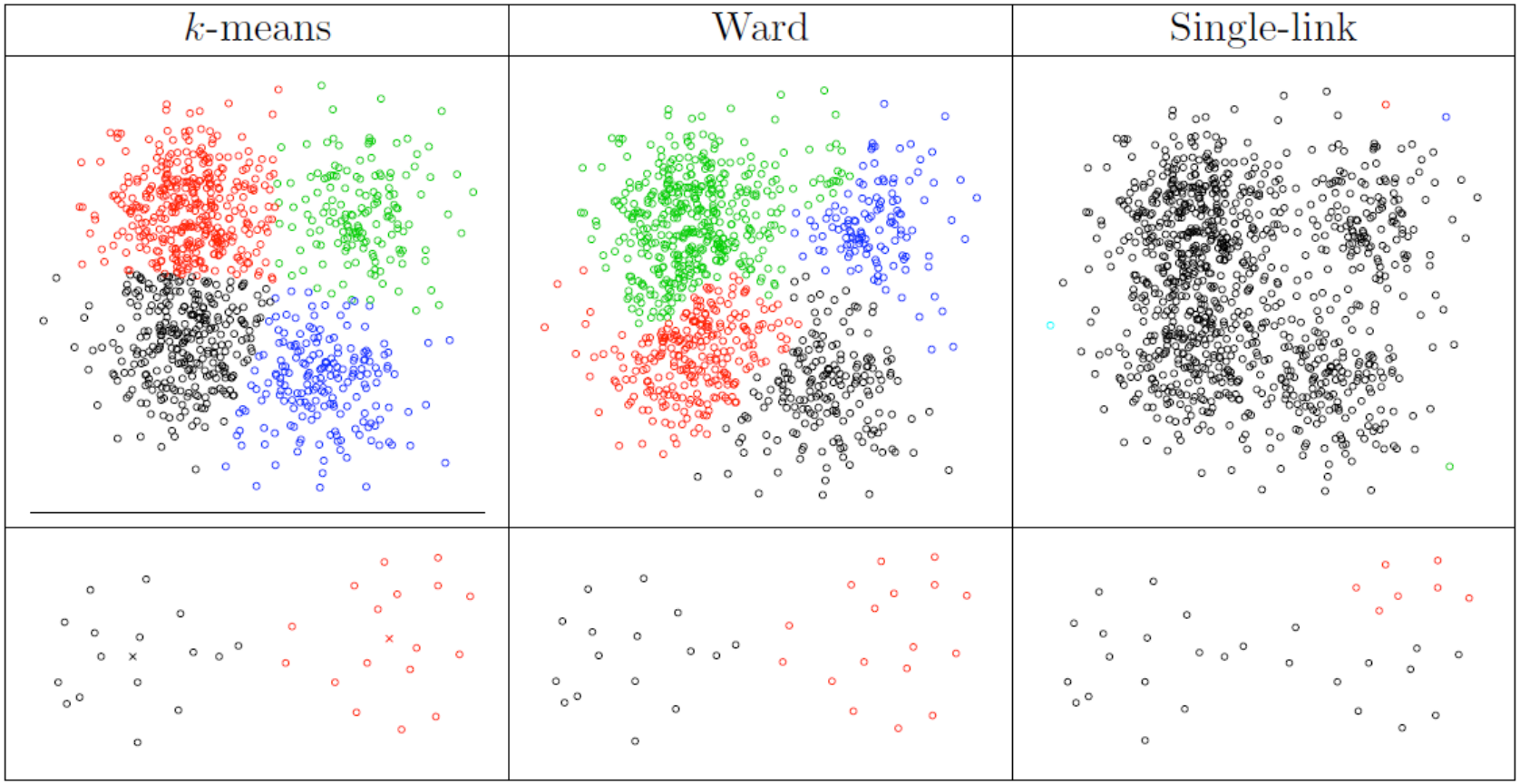
Things to notice:

- If two pairs of clusters have centers that are equally far apart, Ward's method prefers to merge the smaller ones
- At each merge step, Ward's method looks similar to k-means, but constrained by choices in previous iterations
- So the total sum-of-squares for a given k is normally larger for Ward's method than for k-means
- A common trick: Use Ward's method to initialize, then run k-means starting from the Ward clusters

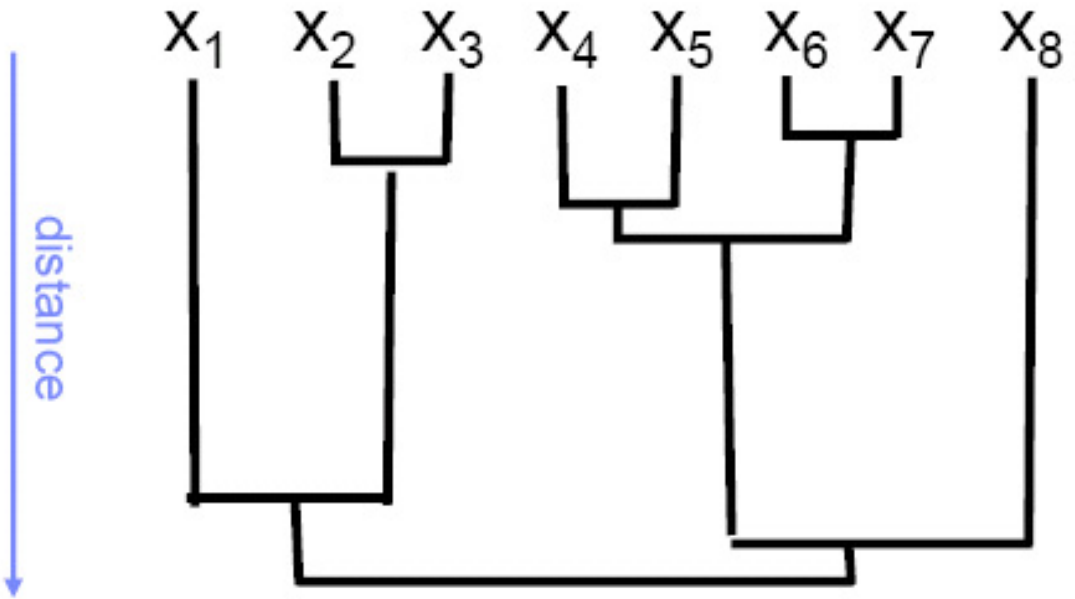
Clustering examples



More clustering examples



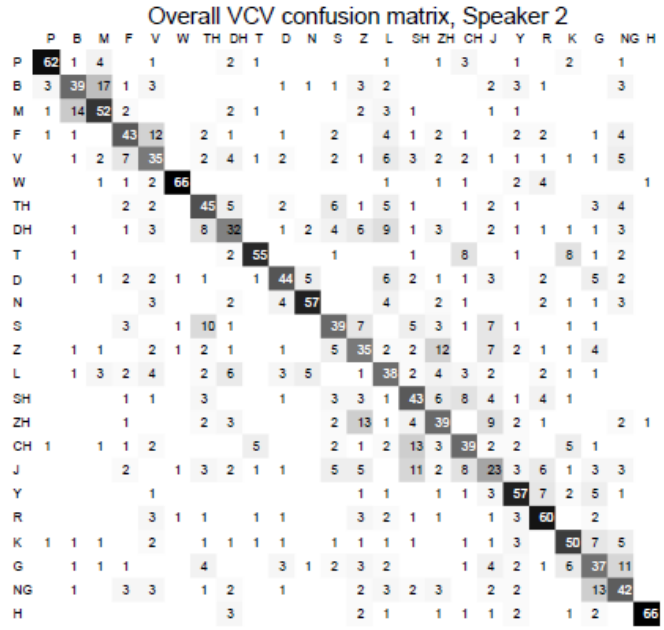
Representing clustering with dendrograms

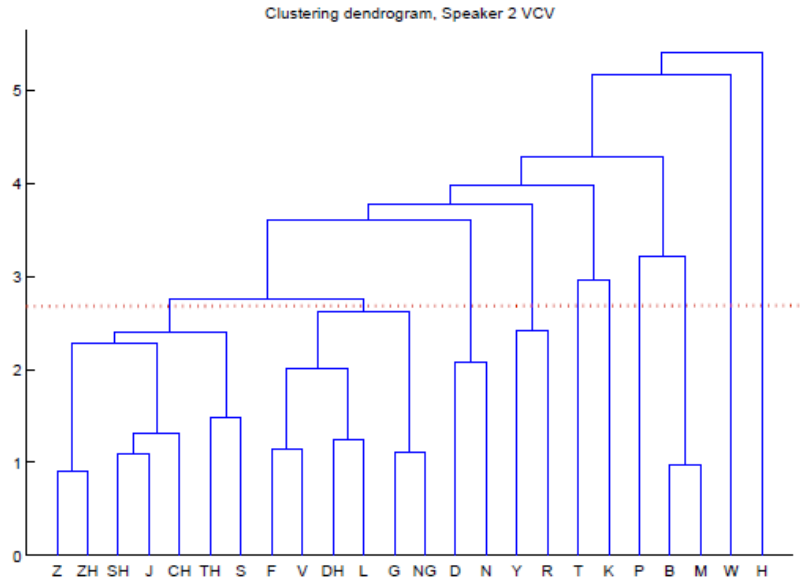


- Items along x-axis, distance along y-axis
- Provides visual guidance to a good choice for the number of clusters
- Quantitative criterion for choosing k: Stop merging when the merge cost (distance between merged clusters) would be much larger than in previous iterations (for some precise definition of "much larger")
- This is one advantage of hierarchical clustering over "flat" clustering like k-means

Example applications

Discovering speech sounds recorded by an "ultrasonic microphone"

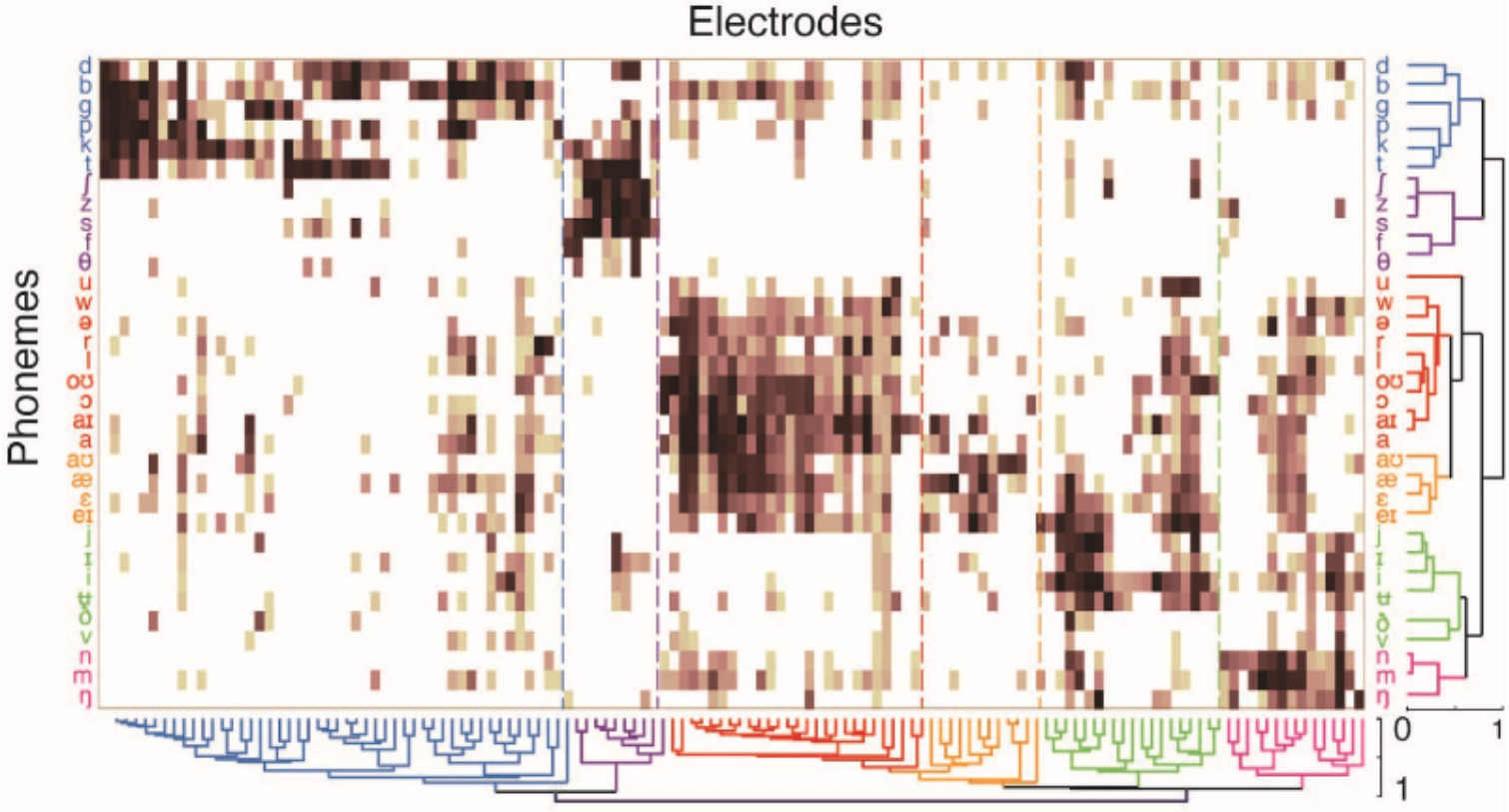




[Livescu *et al.* 2009]

Example applications

Discovering cohesive neural regions/firing patterns and relating them to clusters of speech stimuli



[Bouchard *et al.* 2013]

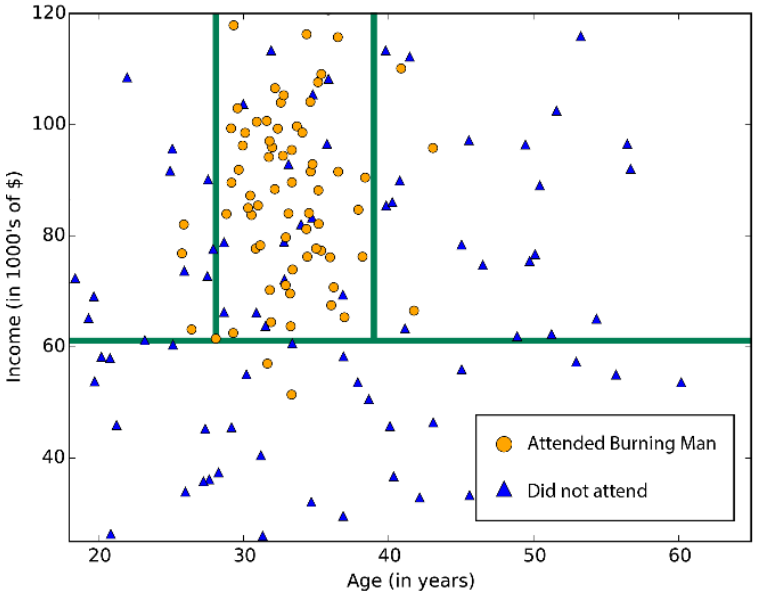
Decision tree clustering

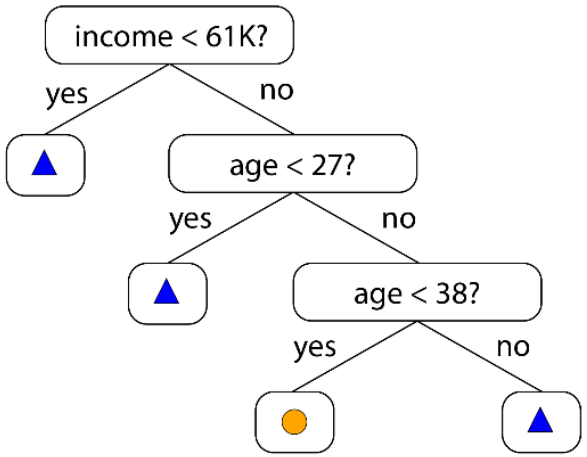
- Agglomerative clustering doesn't get to see all data points at once – might miss some important pattern
- Another hierarchical alternative: top-down clustering using decision trees

Decision tree construction

- Start with all data in a single node (the root)
- Find the best "question" for partitioning the data at a given node into two classes
- Repeat for each node
- Stop when there is insufficient data at each node, or when the best question isn't helpful enough

Decision tree example





Decision tree construction: Design decisions

- Choice of questions for the splits
- Criterion for the "best" question at each node
- Stopping criterion

Decision trees (DTs)

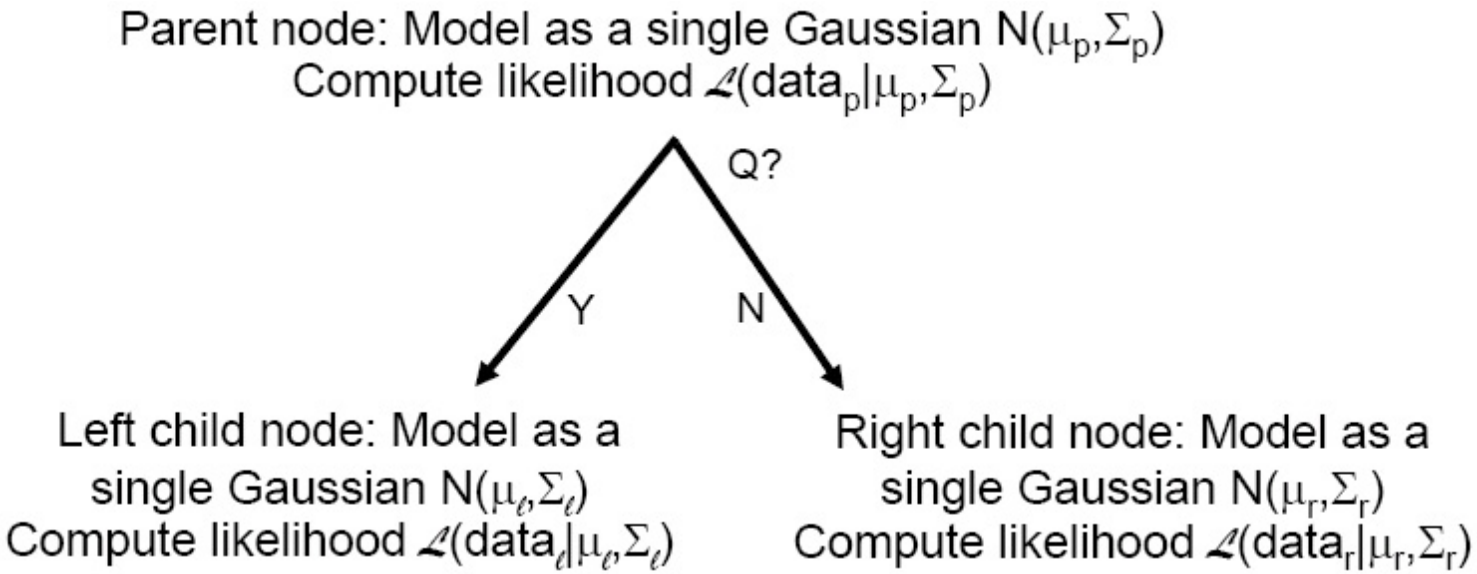
- The trees we are considering are a type of *classification and regression trees* (CART)
- Our DTs are maps from *features* (dimensions of $\mathbf{x}$) to cluster labels
- Each non-leaf node specifies a "question" about the features ($x_1 > 17$? $x_3 = x_{25}$?)
- Each leaf node represents a decision

Overview of tree construction: Example

- Let $x_1, \ldots, x_d$ denote feature dimensions whose values we can query about
- Suppose we have a fixed, discrete set of questions
- Let Q_{ij} denote the jth pre-determined question for x_i
- Starting at the root, try splitting each node into two sub-nodes
  - For each variable x_i, evaluate questions $Q_{i1}, Q_{i2} \ldots$; let x^*, Q_i^* denote the best feature and question
  - If Q_i^* isn't sufficiently helpful, call the current node a leaf
  - Otherwise, split the current node into two sub-nodes according to the answer to question Q_i^*
- Stop when all nodes are either too small to split or have been marked as leaves (there is no "good enough" question anymore)

Evaluating question quality

- Typical criterion: The best question at a node is the *maximum likelihood* one, i.e. the one that maximizes $\mathcal{L}(\text{data}_l|\text{model}_l) \times \mathcal{L}(\text{data}_r|\text{model}_r)$
- Often, the models for the left and right nodes are each a single Gaussian: $Q^* = \text{argmax}_Q\, p(\text{data}_l|\vec{\mu}_l, \vec{\Sigma}_l) \times p(\text{data}_r|\vec{\mu}_r, \vec{\Sigma}_r)$



Evaluating question quality (2)

- Another common criterion: Minimize node *entropy* (uncertainty measured in bits)
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$\begin{aligned} H(X) &= -\sum_{i=1}^{N} p_i \log_2 p_i \\ \Rightarrow H(X) &\geq 0, \\ H(X) &= 0 \Leftrightarrow p_j = 1 \text{ for some } j, p_i = 0 \; \forall i \neq j \end{aligned}$$

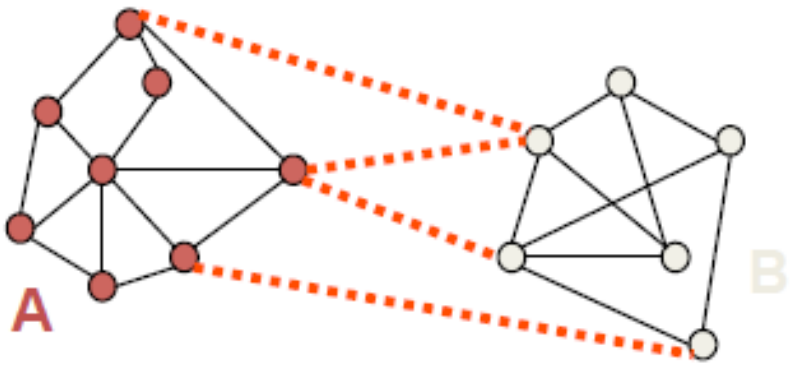
When to stop?

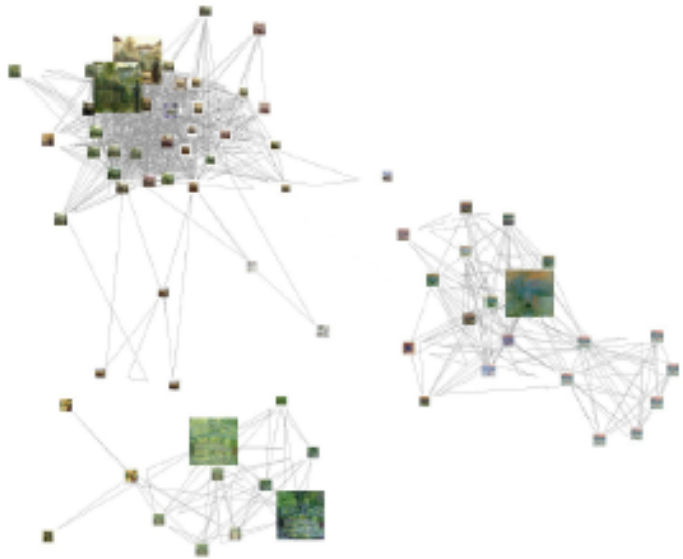
- Cross-validation: Measure likelihood with different tree sizes on a held-out data set, choose the tree size that maximizes likelihood
- In practice, simple heuristics are often used:
  - Data at node has fewer than threshold T samples
  - Best question does not improve likelihood significantly (note: best question should *always* improve likelihood somewhat!)

Spectral clustering

Spectral clustering

- Use a similarity graph (as in graph-based dimensionality reduction)
- Use spectral (eigenvector-based) methods to divide the graph into connected sub-graphs

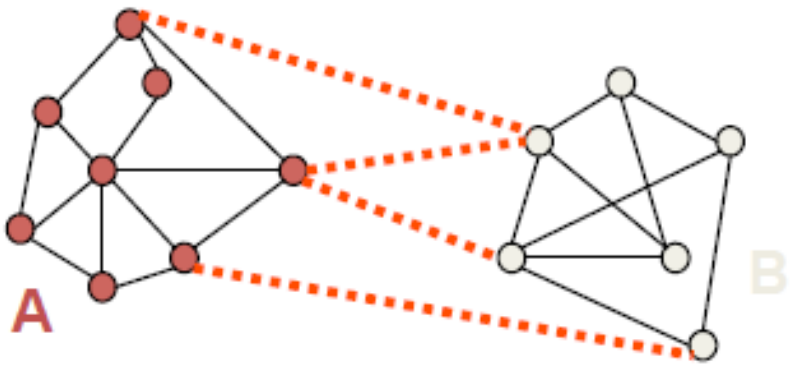


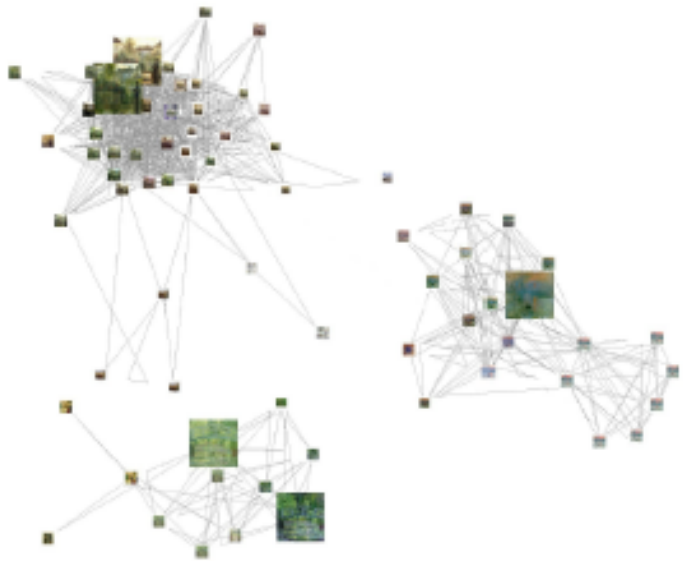


[D. Sontag]

Spectral clustering: preliminaries

- Data $\mathbf{x}_1, \ldots, \mathbf{x}_N$
- Similarities s_{ij}
- Create a graph $G = (V, E)$
  - One node v_i for each $\mathbf{x}_i$
  - Edge weights w_{ij} based on s_{ij}



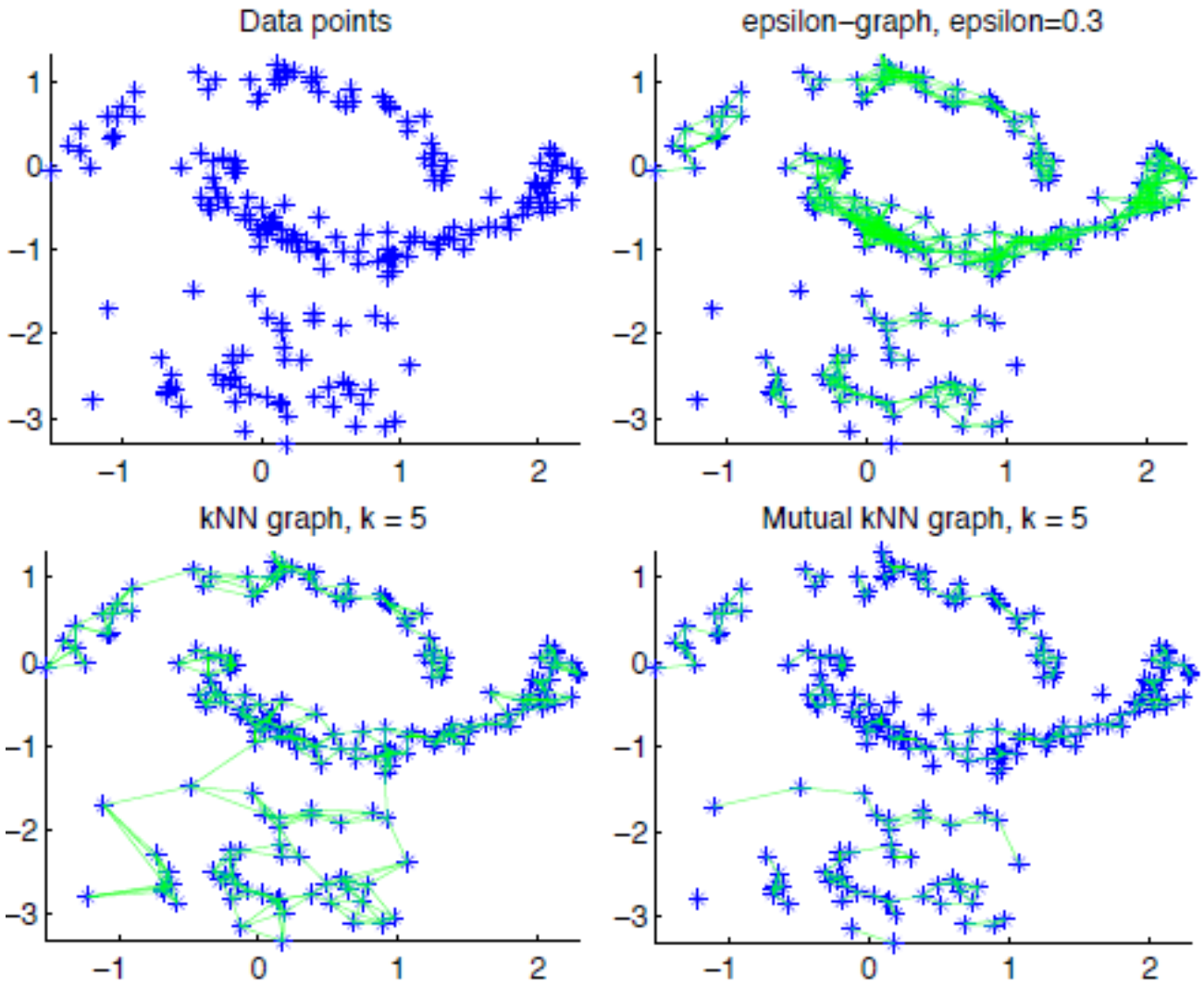


[D. Sontag]

Types of graphs

- Fully connected, weighted by similarity w_{ij}
- ϵ-neighborhood graph: unweighted, include edges with distance $< \epsilon$
- k-NN: include edges where v_i is a k-NN of v_j or vice versa
- Mutual k-NN: include edges where v_i is a k-NN of v_j and vice versa

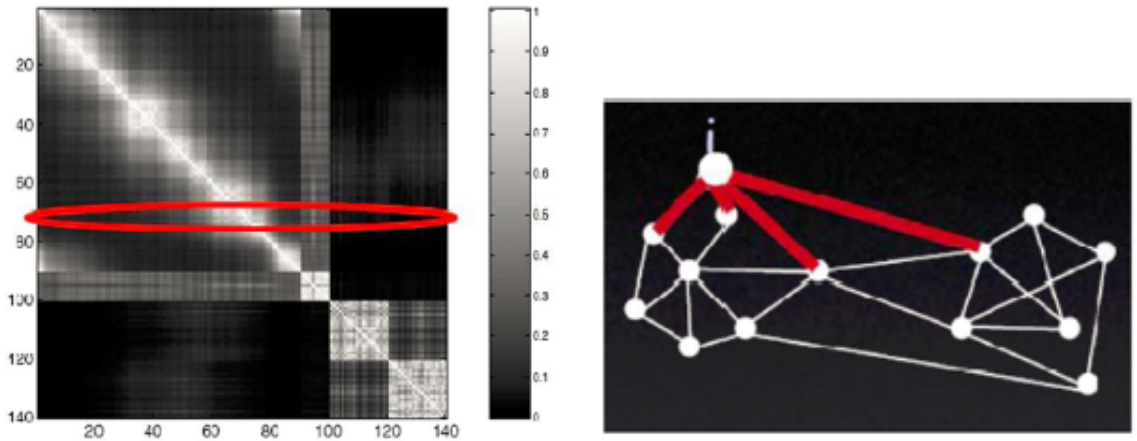
Types of graphs



[von Luxburg]

Adjacency matrices

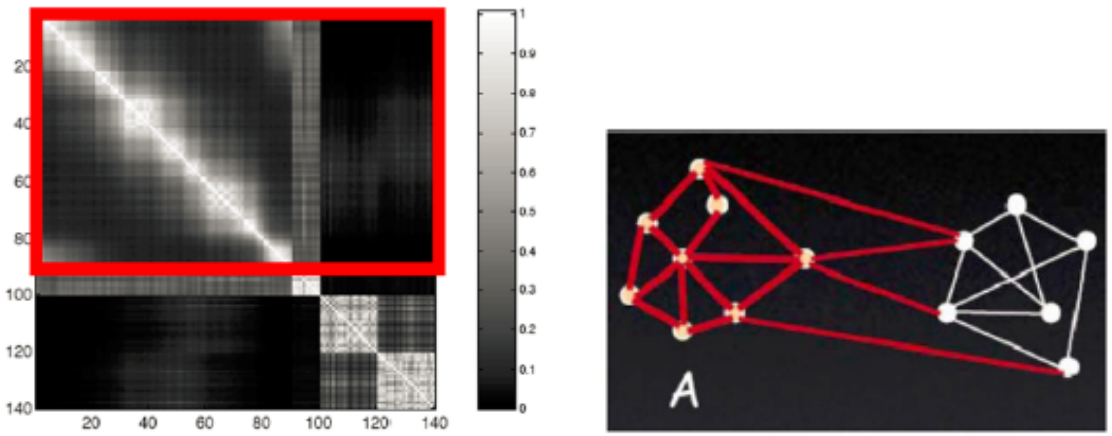
- Adjacency matrix = matrix of edge weights (similarities)
- Can derive quantities of interest from it, e.g.
- Node degree $d_i = \sum_j w_{ij}$



[D. Sontag]

Adjacency matrices

- Adjacency matrix = matrix of edge weights (similarities)
- Can derive quantities of interest from it, e.g.
- Volume of set of nodes $vol(A) = \sum_{i \in A} d_i$

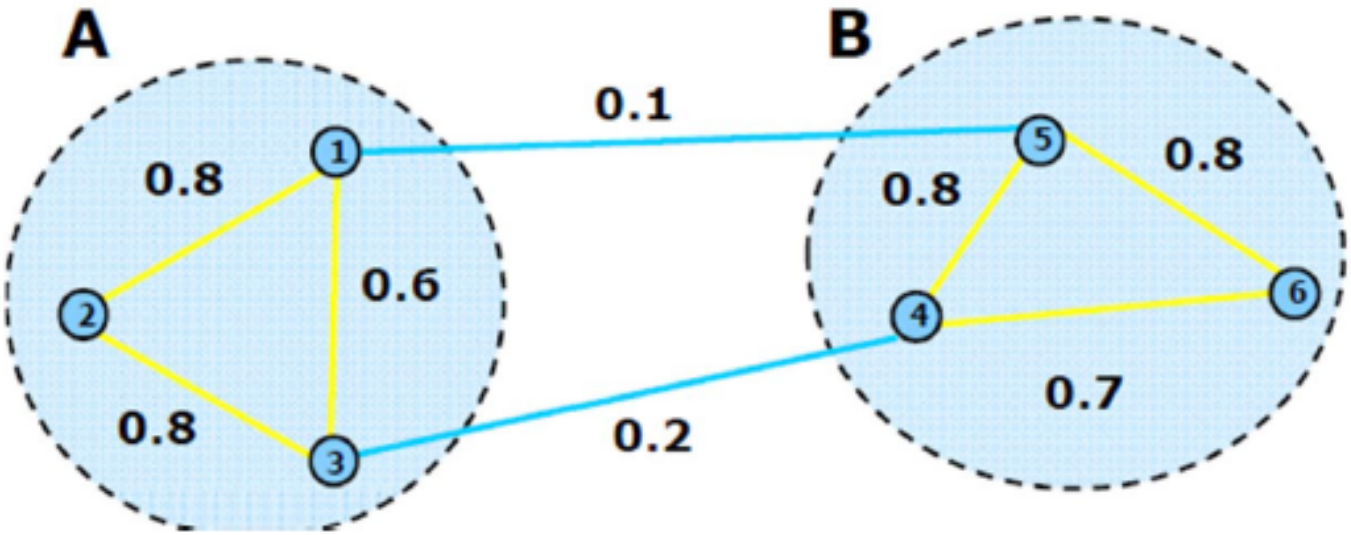


[D. Sontag]

Graph cuts

- Consider some partition of the graph into two sub-graphs **A** and **B**
- $Cut(A, B) = W(A, B) = \sum_{i \in A, j \in B} w_{ij}$ ($= 0.3$ in this case)
- We would like to minimize the cuts between each cluster and the rest of the graph (note: $\bar{A}_i$ is the set of all nodes not in A_i):

$$\underset{A_1,\ldots,A_k}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{k} W(A_i, \bar{A}_i)$$



[Hamad & Biela]

MinCut favors isolated nodes



[E. Fox]

Cuts that account for cluster size

- Ratio cuts (RatioCut) normalizes by cardinality: $\frac{1}{2}\sum_{i=1}^{k}\frac{W(A_i,\bar{A}_i)}{|A_i|}$
- Normalized cuts (Ncut) normalizes by volume: $\frac{1}{2}\sum_{i=1}^{k}\frac{W(A_i,\bar{A}_i)}{vol(A_i)}$



[E. Fox]

Graph Laplacian

Solution is found via a graph Laplacian

- This term has multiple meanings in the literature...
- Let W be the similarity matrix
- Let D be the diagonal degree matrix $D_{ii} = \sum_j W_{ij}$
- Then $L = D - W$ is the unnormalized graph Laplacian

Graph Laplacian properties

- Suppose G is an undirected graph with non-negative weights
- Then 0 is an eigenvalue of L and the multiplicity k of the 0 eigenvalue is the number of connected components in the graph
- So if the clusters are disconnected, easy to tell how many clusters!
- If not, we still use the first k eigenvectors, but it's less obvious why...
  - Graph cuts view
  - Random walks
  - Perturbation theory
  - (See the readings)

Solving normalized cut for 2 clusters

- Define a vector $\mathbf{x} \in \{-1, 1\}^N$, where $x_i = 1$ means data point i is in cluster A; otherwise cluster B
- Can show:

$$\begin{aligned} \min_{\mathbf{x}} \mathrm{Ncut}(\mathbf{x}) \quad &= \quad \min_{\mathbf{y}} \frac{\mathbf{y}^T(D-W)\mathbf{y}}{\mathbf{y}^T D\mathbf{y}} \\ & \quad s.t. \quad \mathbf{y}^T D1 = 0 \end{aligned}$$

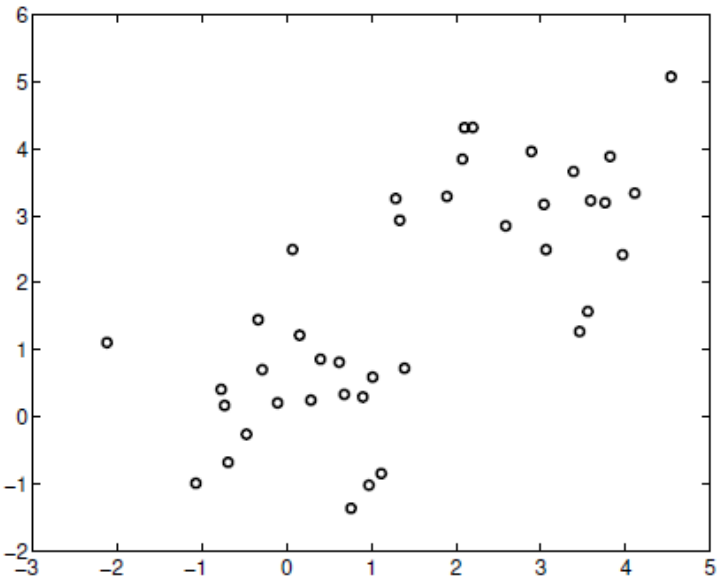
Solving normalized cut for 2 clusters

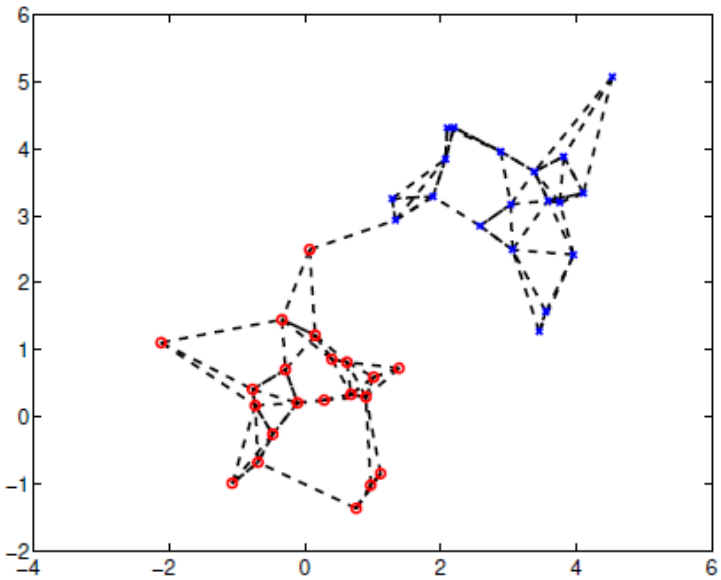
- The optimization problem is relaxed to solve for a continuous vector instead:

$$\begin{aligned} \min_{\mathbf{y}} \quad & \mathbf{y}^T(D-W)\mathbf{y} \\ s.t. \quad & \mathbf{y}^T D\mathbf{y} = 1 \\ \Longrightarrow \quad & (D-W)\mathbf{y} = \lambda D\mathbf{y} \end{aligned}$$

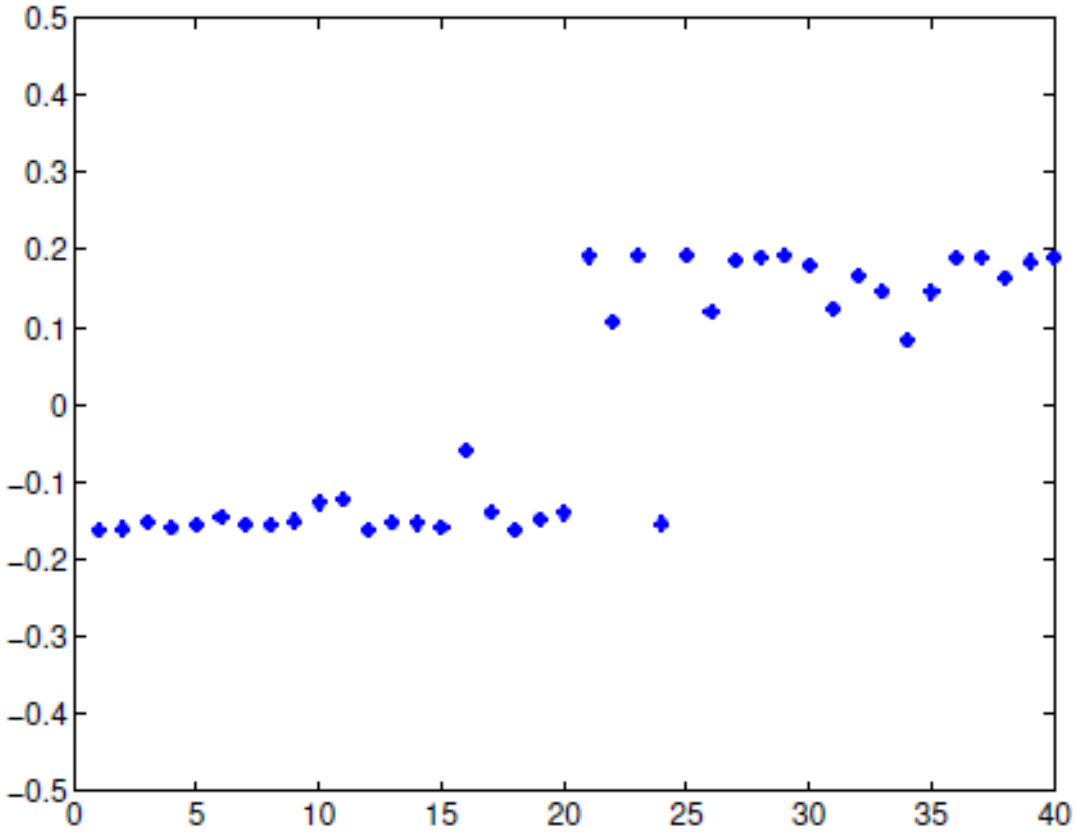
- The first eigenvector is all ones (all data in single cluster)
- We take the 2nd eigenvector as the real-valued solution

Example





Example: The 2nd eigenvector

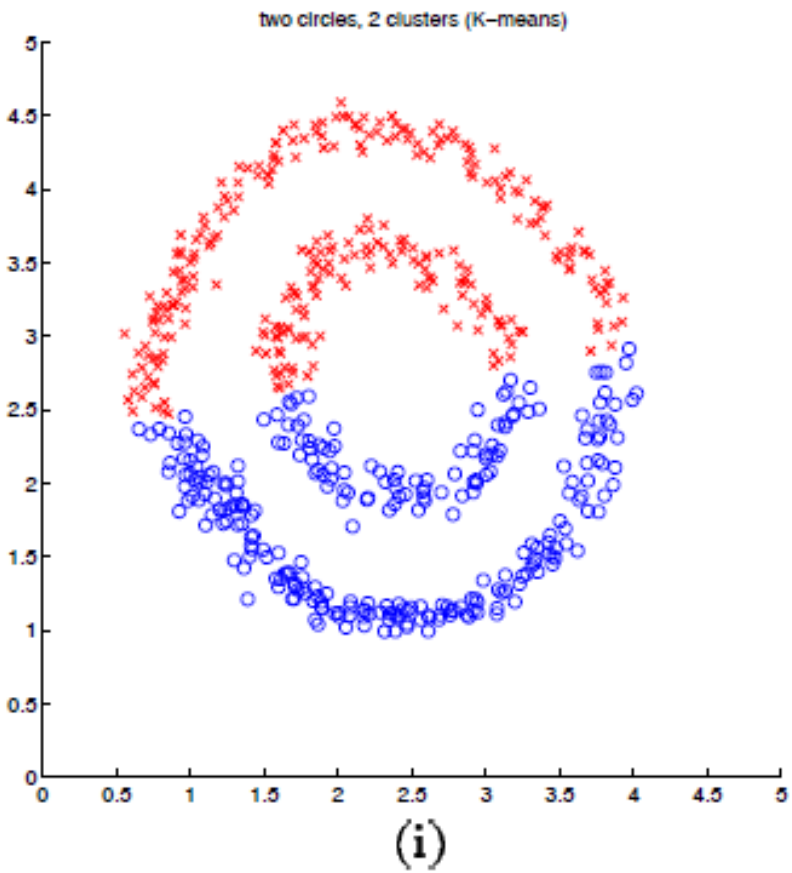


Beyond 2 clusters

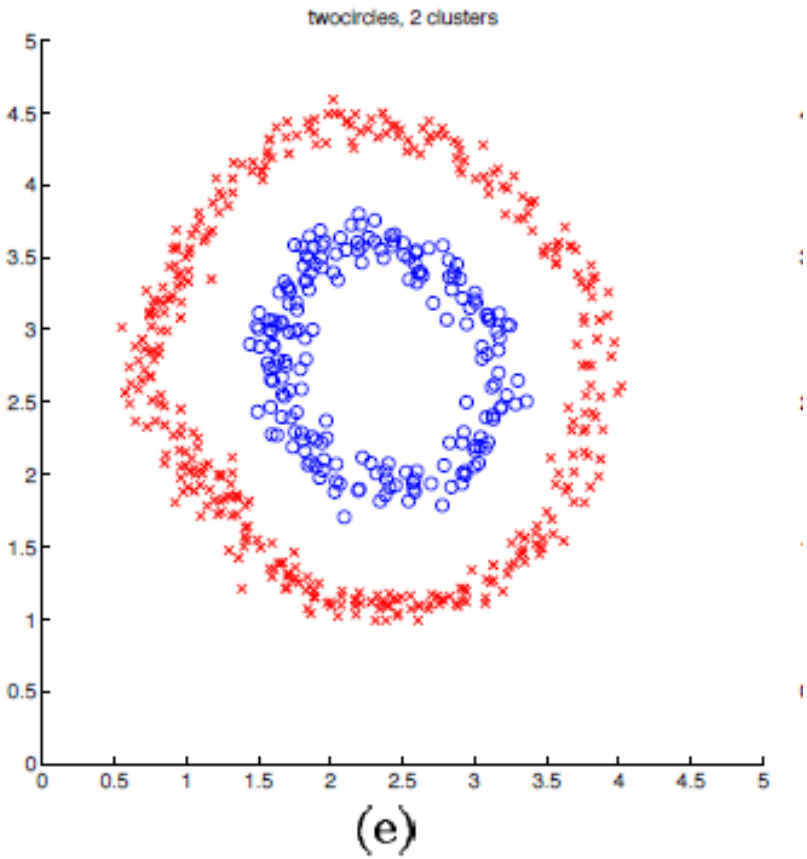
- Option 1: Recursively apply the 2-cluster algorithm
- Option 2: Treat the first k eigenvectors as a reduced-dimensionality representation of the data, and cluster the eigenvectors

Example: K-means vs. Spectral clustering

K-means

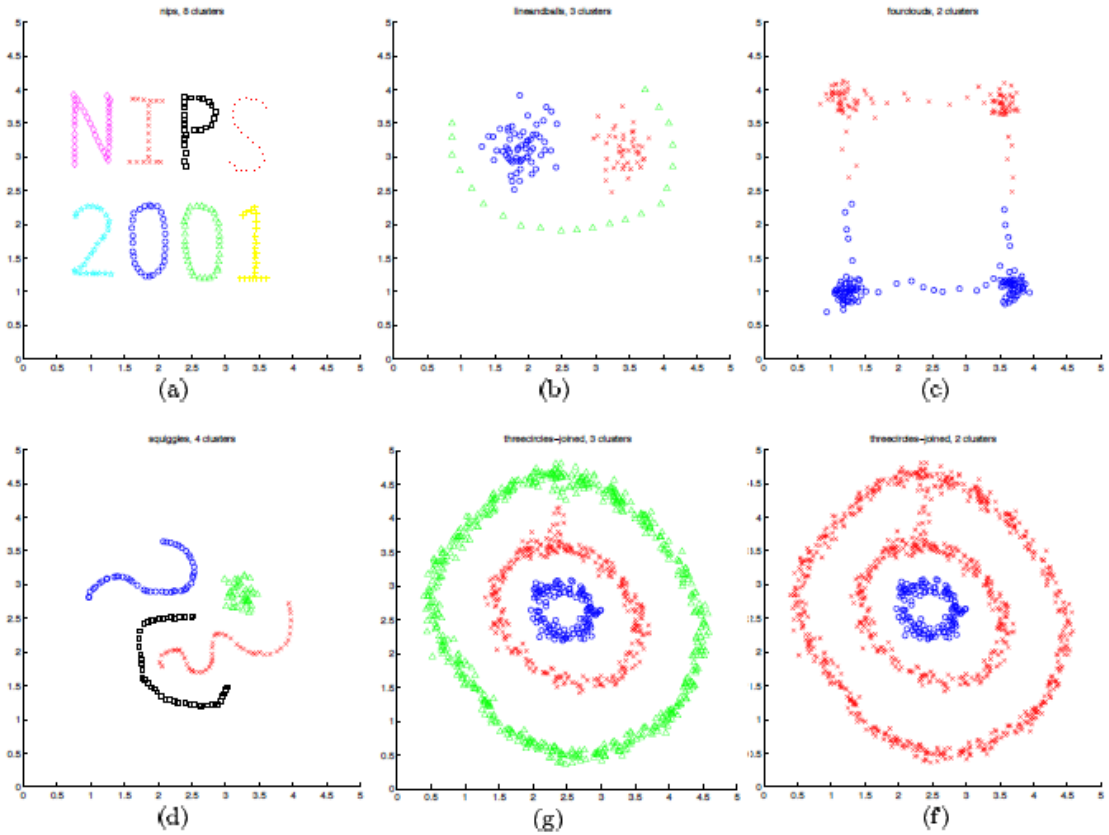


Spectral clustering



[Ng, Jordan, & Weiss]

Example: Spectral clustering

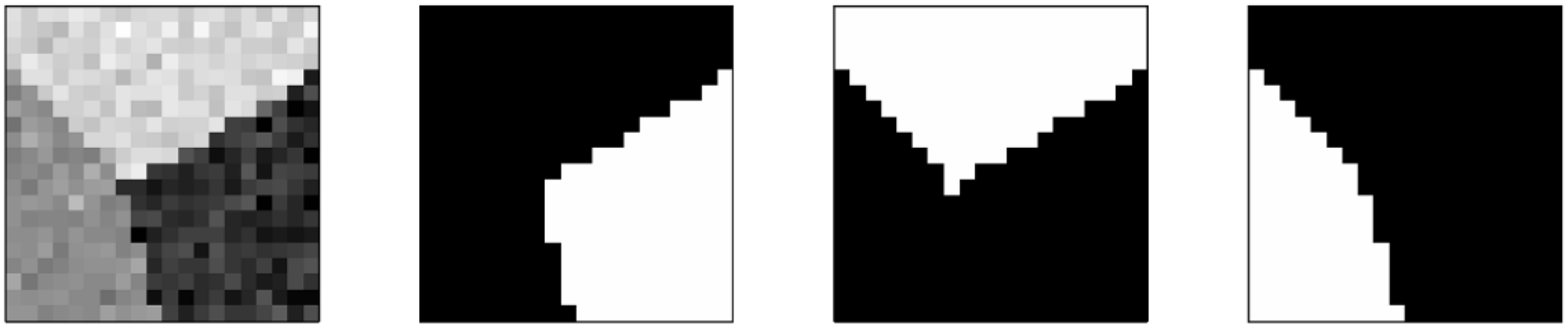


[Ng, Jordan, & Weiss]

Spectral clustering: A real (but old) application

Image segmentation: Each pixel i is a graph vertex, similarity depends on pixel content (color intensity) $F(i)$ and pixel location $X(i)$:

$$w_{ij} = e^{-\left(\frac{1}{2\pi\sigma^2}||F(i)-F(j)||^2+\frac{1}{2\pi\tau^2}||X(i)-X(j)||^2\right)}, \text{ if } ||X(i)-X(j)|| < r \text{ ; } 0 \text{ o.w.}$$



(a) (b) (c) (d)

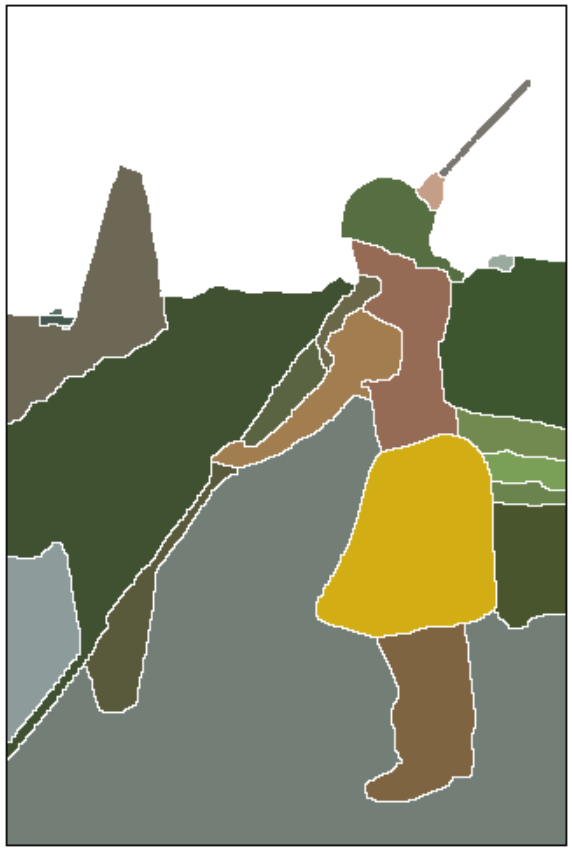
[Shi & Malik]

Spectral clustering: A real (but old) application

Image segmentation: Each pixel i is a graph vertex, similarity depends on pixel content (color intensity) $F(i)$ and pixel location $X(i)$:

$$w_{ij} = e^{-\left(\frac{1}{2\pi\sigma^2}||F(i)-F(j)||^2+\frac{1}{2\pi\tau^2}||X(i)-X(j)||^2\right)}, \text{ if } ||X(i)-X(j)|| < r \text{ ; } 0 \text{ o.w.}$$

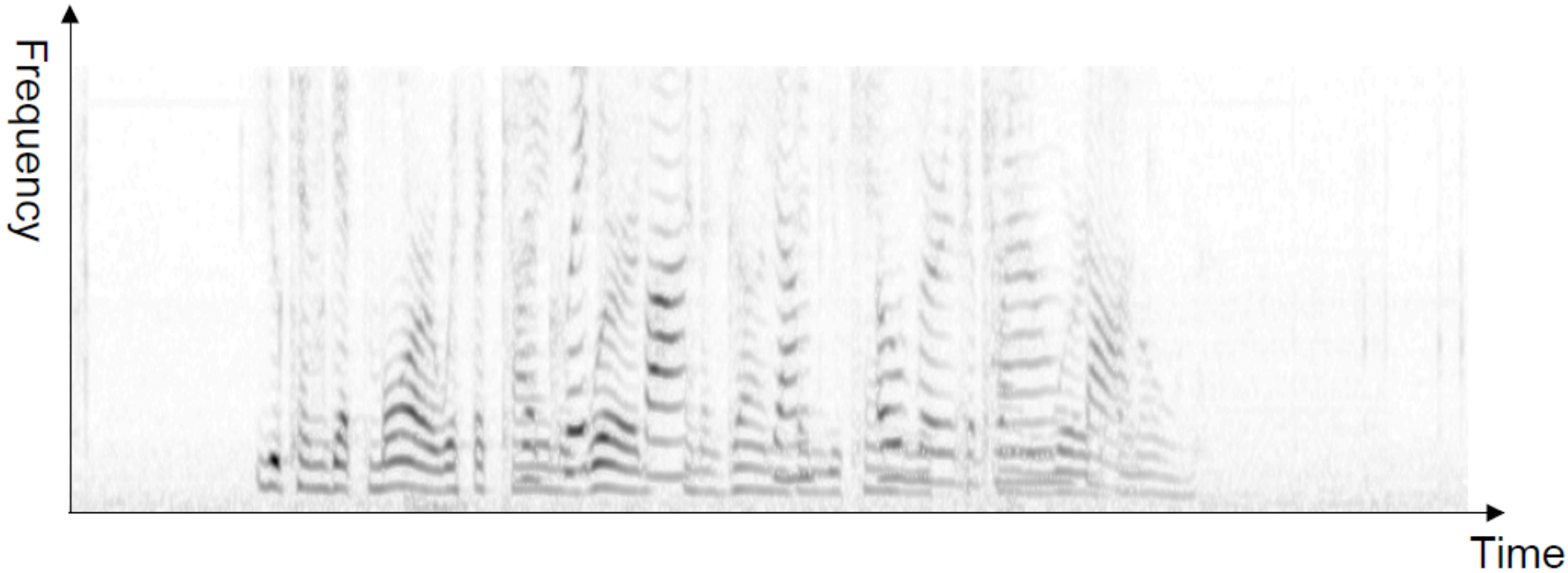




[Arbeláez *et al.*]

Spectral clustering: A real (slightly less old) application

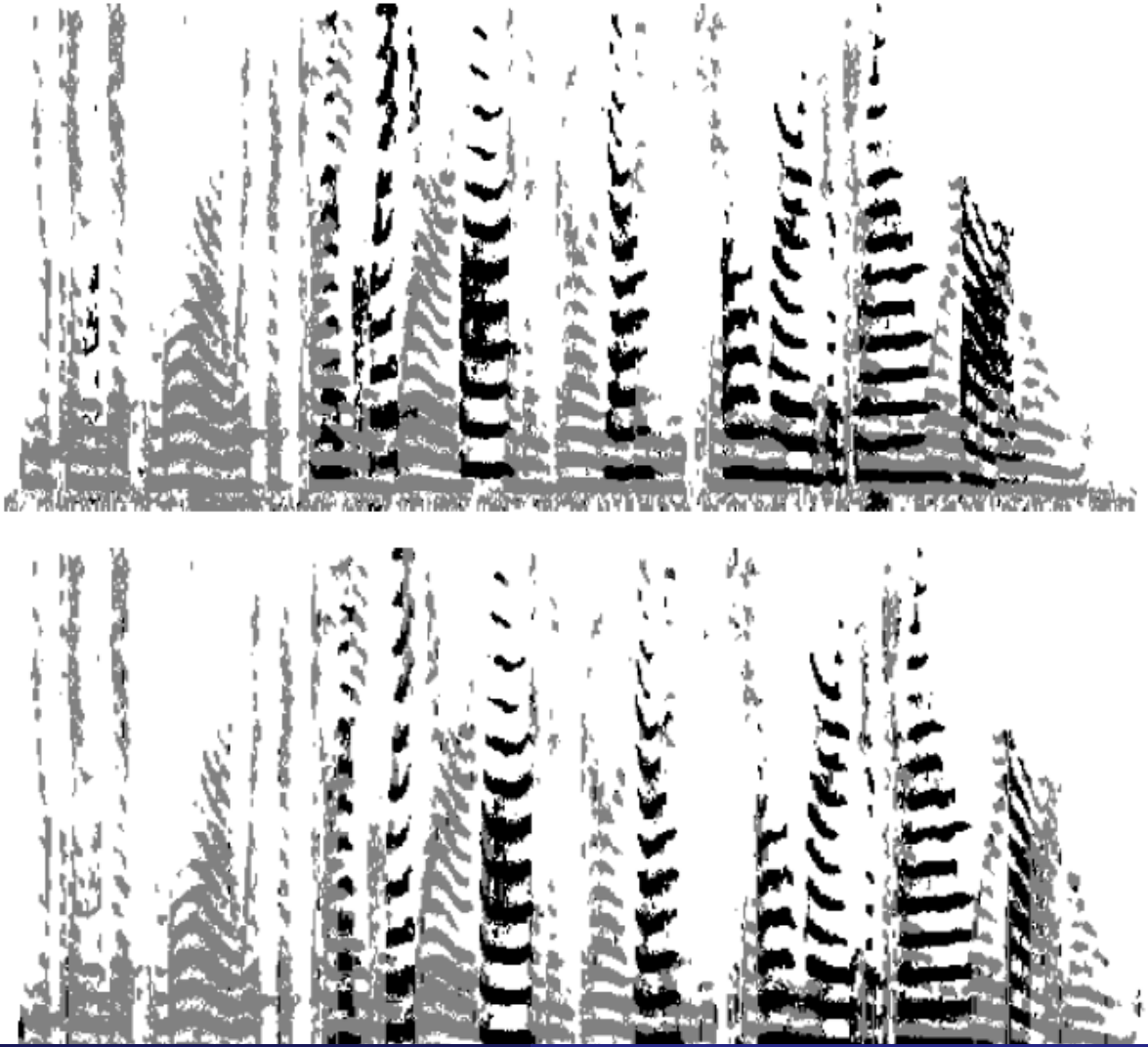
Speech separation (into speakers): Similar to image segmentation, where a "pixel" is a cell of a spectrogram:



[Bach & Jordan]

Spectral clustering: A real (slightly less old) application

Speech segmentation into speakers (top = true; bottom = predicted):



From clustering to density estimation: Gaussian mixtures

From clustering to mixture models

Clustering makes a hard decision about cluster membership for each data point

- Can't always do out-of-sample extension
  - K-means: Can map new points to cluster with nearest cluster mean
  - No obvious way to do this for hierarchical clustering, spectral clustering

From clustering to mixture models

Clustering makes a hard decision about cluster membership for each data point

- Can't always do out-of-sample extension
  - K-means: Can map new points to cluster with nearest cluster mean
  - No obvious way to do this for hierarchical clustering, spectral clustering

Mixture models represent the density of the data as a mixture of component densities

- Can be seen as "soft" clustering
- Hard cluster ID can be assigned if needed
- New data points (soft-)assigned to clusters naturally
- We'll start with mixtures of Gaussians

Recall: What's good about Gaussian distributions?

- Arbitrary linear combinations of jointly Gaussian variables are also Gaussian
- Marginals and conditionals of multivariate Gaussians are also Gaussian
- Sums of many i.i.d. random variables converge to Gaussian variables (Central Limit Theorem)
- Can model non-Gaussian distributions as mixtures of Gaussians
- Log of Gaussian looks like a weighted Euclidean distance
  $-\ln\sqrt{(2\pi\sigma)} - \frac{(\mathbf{x}-\mu)^2}{(2\sigma^2)}$

Maximum-likelihood (ML) estimation of Gaussian parameters

Given data $X = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(X|\mu, \Sigma) = -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln |\Sigma| - \frac{1}{2} \sum_{n=1}^{N} (\mathbf{x}_n - \mu)^T \Sigma^2 (\mathbf{x}_n - \mu)$$

Maximum-likelihood (ML) estimation of Gaussian parameters

Given data $X = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(X|\mu, \Sigma) = -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln |\Sigma| - \frac{1}{2} \sum_{n=1}^{N} (\mathbf{x}_n - \mu)^T \Sigma^2 (\mathbf{x}_n - \mu)$$

Setting the derivative to zero:

$$\frac{\partial}{\partial \mu} \ln p(X|\mu, \Sigma) = \sum_{n=1}^{N} \Sigma^{-1} (\mathbf{x}_n - \mu) = 0$$

which gives

$$\mu_{ML} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n$$

Maximum-likelihood (ML) estimation of Gaussian parameters

Given data $X = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(X|\mu, \Sigma) = -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln |\Sigma| - \frac{1}{2} \sum_{n=1}^{N} (\mathbf{x}_n - \mu)^T \Sigma^2 (\mathbf{x}_n - \mu)$$

Setting the derivative to zero:

$$\frac{\partial}{\partial \mu} \ln p(X|\mu, \Sigma) = \sum_{n=1}^{N} \Sigma^{-1} (\mathbf{x}_n - \mu) = 0$$

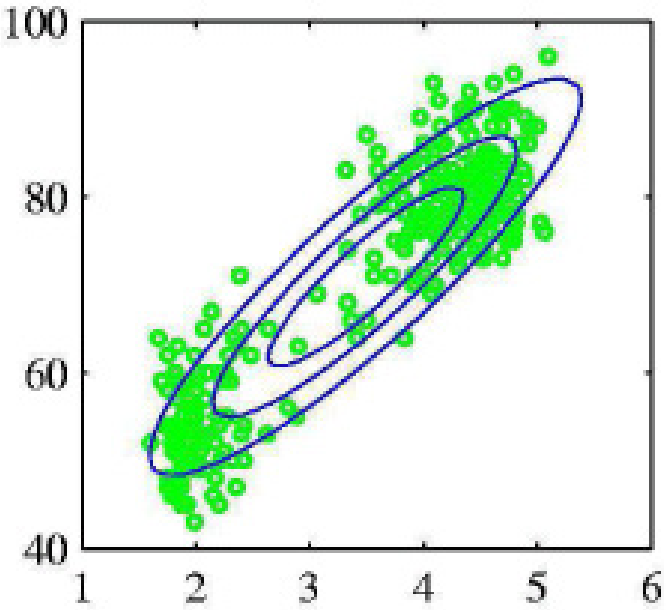
which gives

$$\mu_{ML} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n$$

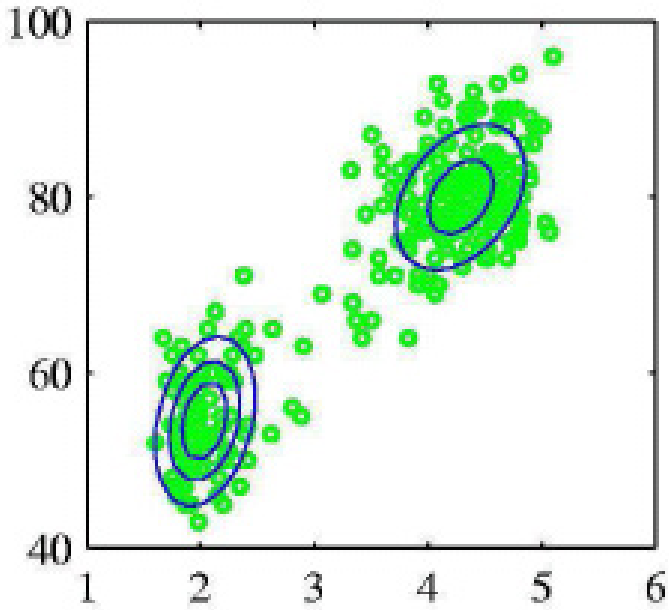
Similarly,

$$\Sigma_{ML} = \frac{1}{N} \sum_{n=1}^{N} (x_n - \mu_{ML})(x_n - \mu_{ML})^T$$

Gaussian mixtures (mixture of Gaussians, GMM)

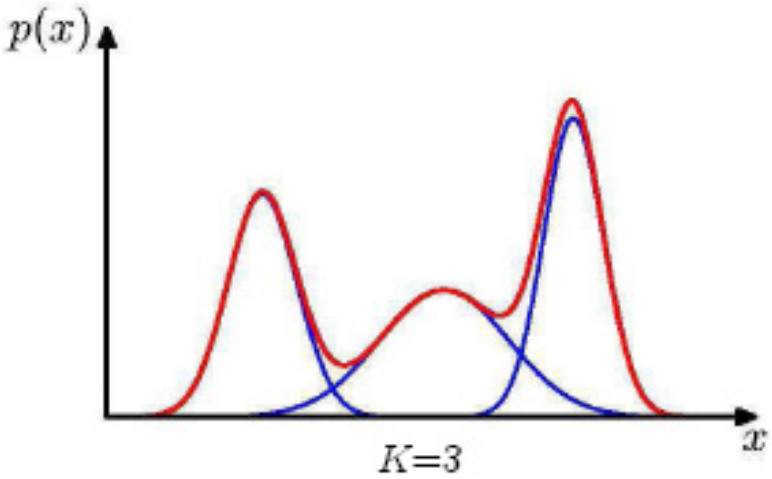


Single Gaussian



Mixture of two Gaussians

Gaussian mixtures



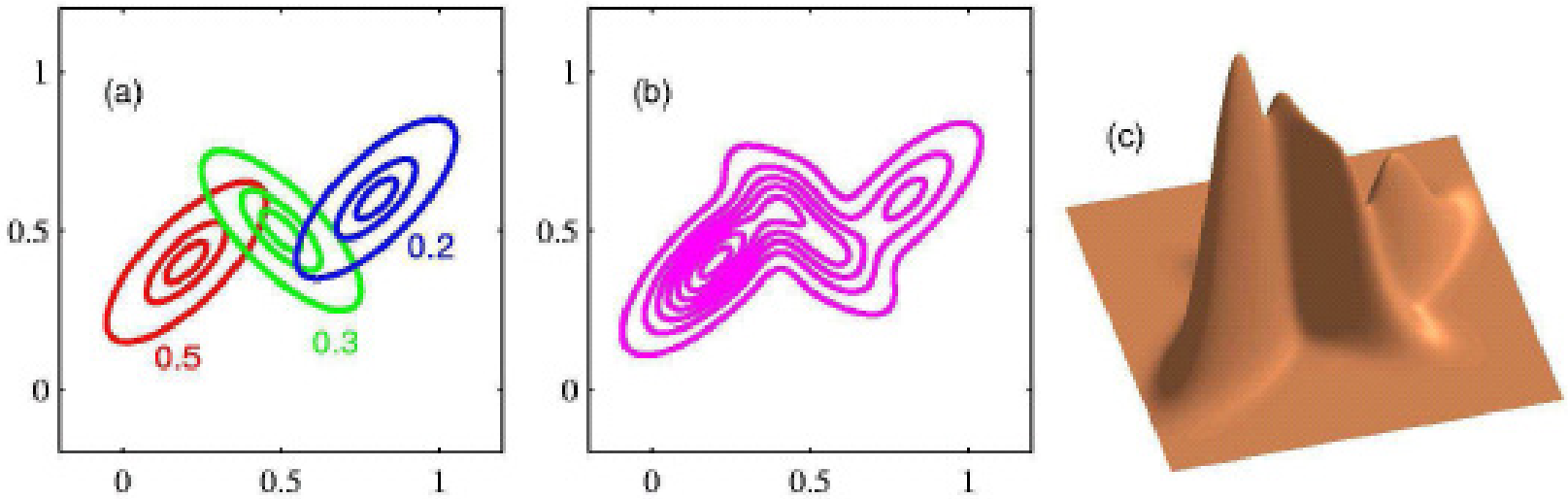
$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Component

Mixing coefficient

$$\forall k : \pi_k \geqslant 0 \qquad \sum_{k=1}^{K} \pi_k = 1$$

Gaussian mixtures



- Essentially any distribution can be approximated arbitrarily well by a large enough Gaussian mixture
- But, maximum-likelihood estimation of parameters is not trivial

Gaussian mixtures: Issues with ML estimation

For given data set $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, want to maximize

$$\ln p(X|\pi, \theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k)$$

No closed-form solution as in single-Gaussian case

- We'll deal with this issue in detail next

Gaussian mixtures: Issues with ML estimation

For given data set $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, want to maximize

$$\ln p(X|\pi, \theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k)$$

No closed-form solution as in single-Gaussian case

- We'll deal with this issue in detail next

Other issues (will get back to them later)

- What if we set $\mu_1 = \mathbf{x}_1$, let $\sigma_1 \to 0$ for one of the Gaussians?

Gaussian mixtures: Issues with ML estimation

For given data set $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, want to maximize

$$\ln p(X|\pi, \theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k)$$

No closed-form solution as in single-Gaussian case

- We'll deal with this issue in detail next

Other issues (will get back to them later)

- What if we set $\mu_1 = \mathbf{x}_1$, let $\sigma_1 \to 0$ for one of the Gaussians?
- $\Longrightarrow$ need *regularization*

Gaussian mixtures: Issues with ML estimation

For given data set $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, want to maximize

$$\ln p(X|\pi, \theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k)$$

No closed-form solution as in single-Gaussian case

- We'll deal with this issue in detail next

Other issues (will get back to them later)

- What if we set $\mu_1 = \mathbf{x}_1$, let $\sigma_1 \to 0$ for one of the Gaussians?
- $\Longrightarrow$ need *regularization*
- How many Gaussians K?

Gaussian mixtures: Issues with ML estimation

For given data set $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, want to maximize

$$\ln p(X|\pi, \theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k)$$

No closed-form solution as in single-Gaussian case

- We'll deal with this issue in detail next

Other issues (will get back to them later)

- What if we set $\mu_1 = \mathbf{x}_1$, let $\sigma_1 \to 0$ for one of the Gaussians?
- $\Longrightarrow$ need *regularization*
- How many Gaussians K?
- What if we pick K by maximizing likelihood?

Gaussian mixtures: Issues with ML estimation

For given data set $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, want to maximize

$$\ln p(X|\pi, \theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k)$$

No closed-form solution as in single-Gaussian case

- We'll deal with this issue in detail next

Other issues (will get back to them later)

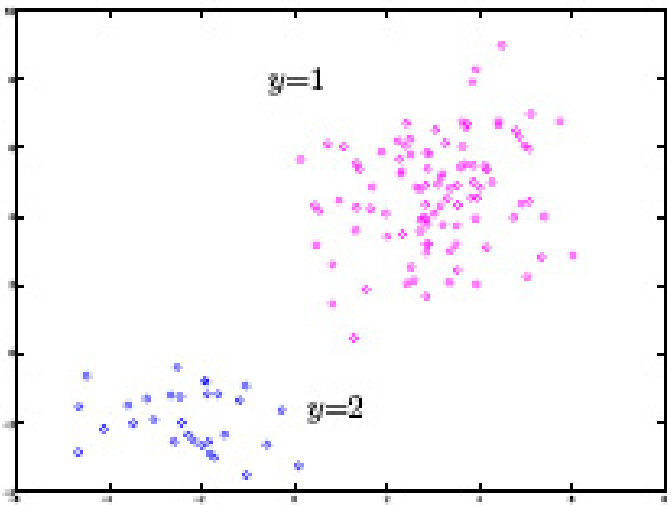
- What if we set $\mu_1 = \mathbf{x}_1$, let $\sigma_1 \to 0$ for one of the Gaussians?
- $\Longrightarrow$ need *regularization*
- How many Gaussians K?
- What if we pick K by maximizing likelihood?
- What if we have one Gaussian for each data point?

Gaussian mixtures: ML estimation

- Introduce a set of binary indicator variables $z_{i1}, \ldots, z_{iK}$, where $z_{ik} = 1$ if x_i came from Gaussian component k and $z_{ik} = 0$ otherwise
- Count of examples from k^{th} component: $N_k = \sum_{i=1}^{N} z_{ik}$

ML estimation with known component labels

- If we know z_i, then the ML estimates of the Gaussian components are just like in the single-Gaussian case:



$$\hat{\pi}_k = \frac{N_k}{N}$$

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i=1}^{N} z_{ik} x_i$$

$$\hat{\Sigma}_k = \frac{1}{N_k} \sum_{i=1}^{N} z_{ik} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

ML estimation with unknown component labels: The credit assignment problem

- When we don't know $\mathbf{z}_i$, we face a credit assignment problem: Which component was reponsible for $\mathbf{x}_i$?

ML estimation with unknown component labels: The credit assignment problem

- When we don't know $\mathbf{z}_i$, we face a credit assignment problem: Which component was reponsible for $\mathbf{x}_i$?
- Suppose we *do* know the component parameters $\theta = \{\mu_k, \Sigma_k\}$
- Then the *posterior probability* of the indicator variables is (using Bayes' rule)

$$\gamma_{ik} = P(z_{ik} = 1 | \mathbf{x}_i, \theta) = \frac{\pi_k p(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_{l=1}^{K} \pi_l p(\mathbf{x}_i | \mu_l, \Sigma_l)}$$

ML estimation with unknown component labels: The credit assignment problem

- When we don't know $\mathbf{z}_i$, we face a credit assignment problem: Which component was reponsible for $\mathbf{x}_i$?
- Suppose we *do* know the component parameters $\theta = \{\mu_k, \Sigma_k\}$
- Then the *posterior probability* of the indicator variables is (using Bayes' rule)

$$\gamma_{ik} = P(z_{ik} = 1 | \mathbf{x}_i, \theta) = \frac{\pi_k p(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_{l=1}^{K} \pi_l p(\mathbf{x}_i | \mu_l, \Sigma_l)}$$

- γ_{ik} is called the *responsibility* of the kth component for $\mathbf{x}_i$. Note that $\sum_{k=1}^{K} \gamma_{ik} = 1$

ML estimation with unknown component labels

Now, "pretend" that the γ_{ik} are the indicator variables (instead of z_{ik}), i.e., that $N_k = \sum_{i=1}^{N} \gamma_{ik}$:

ML estimation with unknown component labels

Now, "pretend" that the γ_{ik} are the indicator variables (instead of z_{ik}), i.e., that $N_k = \sum_{i=1}^N \gamma_{ik}$:

$$\hat{\pi}_k = \frac{\sum_{i=1}^N \gamma_{ik}}{N}$$

ML estimation with unknown component labels

Now, "pretend" that the γ_{ik} are the indicator variables (instead of z_{ik}), i.e., that $N_k = \sum_{i=1}^N \gamma_{ik}$:

$$\begin{aligned} \hat{\pi}_k &= \frac{\sum_{i=1}^N \gamma_{ik}}{N} \\ \hat{\mu}_k &= \frac{1}{\sum_{i=1}^N \gamma_{ik}} \sum_{i=1}^N \gamma_{ik} \mathbf{x}_i \end{aligned}$$

ML estimation with unknown component labels

Now, "pretend" that the γ_{ik} are the indicator variables (instead of z_{ik}), i.e., that $N_k = \sum_{i=1}^N \gamma_{ik}$:

$$\begin{aligned}
\hat{\pi}_k &= \frac{\sum_{i=1}^N \gamma_{ik}}{N} \\
\hat{\mu}_k &= \frac{1}{\sum_{i=1}^N \gamma_{ik}} \sum_{i=1}^N \gamma_{ik} \mathbf{x}_i \\
\hat{\Sigma}_k &= \frac{1}{\sum_{i=1}^N \gamma_{ik}} \sum_{i=1}^N \gamma_{ik} (\mathbf{x}_i - \hat{\mu}_k)(\mathbf{x}_i - \hat{\mu}_k)^T
\end{aligned}$$

ML estimation with unknown component labels

- What are we maximizing?

ML estimation with unknown component labels

- What are we maximizing?
- Would like to maximize the "complete data" likelihood:

$$p(X, Z|\pi, \theta) \propto \prod_{i=1}^{N} \prod_{k=1}^{K} (\pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k))^{z_{ik}}$$

- Or its log:

$$\ln p(X, Z|\pi, \theta) = \text{const} + \sum_{i=1}^{N} \sum_{k=1}^{K} z_{ik} (\ln \pi_k + \ln \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k))$$

ML estimation with unknown component labels

- What are we maximizing?
- Would like to maximize the "complete data" likelihood:

$$p(X, Z|\pi, \theta) \propto \prod_{i=1}^{N} \prod_{k=1}^{K} (\pi_k \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k))^{z_{ik}}$$

- Or its log:

$$\ln p(X, Z|\pi, \theta) = \text{const} + \sum_{i=1}^{N} \sum_{k=1}^{K} z_{ik} (\ln \pi_k + \ln \mathcal{N}(\mathbf{x}_i|\mu_k, \Sigma_k))$$

- What we are actually maximizing is its expectation w.r.t. $\mathbf{z}$:
  $E_{z_{ik}|X,\pi,\theta}[\ln p(X, Z|\pi, \theta)]$

Summary so far

- If we know the *parameters* and *indicators* (component labels) then we are done
- If we know the *indicators* but not the parameters, we can do ML estimation of the parameters – and we are done
- If we know the *parameters* but not the indicators, we can compute the posteriors of indicators; then estimate parameters that maximize the expected likelihood – and then we are done
- In reality, we know neither the parameters nor the indicators